



UNIVERSITÀ DEGLI STUDI DI CATANIA

DIPARTIMENTO DI MATEMATICA E INFORMATICA

CORSO DI LAUREA MAGISTRALE IN INFORMATICA

Petronio Petronio Davide

Blockchain & CryptoCurrencies

APPUNTI LEZIONI

Prof. Catalano & Prof. Di Raimondo

Anno Accademico 2025 - 2026

Indice

1	Funzioni Hash	5
1.1	Hash Pointers e Struttura della Blockchain	6
1.1.1	Il Merkle Tree	7
1.2	Firma digitale	8
1.3	Basic Cryptocurrencies	9
1.3.1	GoofyCoin	9
1.3.2	ScroogeCoin	10
2	Come Bitcoin ottiene la Decentralizzazione	12
2.1	Le domande fondamentali del sistema	12
2.2	I tre pilastri della decentralizzazione	13
2.3	Il Consenso Distribuito	14
2.3.1	Incentivi e Proof of Work	16
2.4	Struttura delle Transazioni e Modello UTXO	20
3	Consenso di Nakamoto	22
3.1	Dimostrazione Matematica dell'attacco	24
3.1.1	La scelta della Difficoltà e il Network Delay	25
3.1.1.1	Il Discount Rate della potenza onesta	26
3.1.2	Vantaggi e Svantaggi	27
4	Anonimato e Privacy	29
4.1	Perché vogliamo l'anonimato?	30
4.1.1	Anonymous e-cash e Blind Signatures	31
4.1.2	Anonimato grazie a nuovi indirizzi	32
4.2	De-anonimizzazione a livello Network	34
4.2.1	Rete Tor	34
4.3	Mixing e Anonimato nelle Criptovalute	35
4.3.1	Online Wallet	36
4.3.2	Principi di Mixing	37
4.4	Coinjoin	38

5	Zero Knowledge Proofs	40
5.1	Prove Interattive	40
5.1.1	Isomorfismo tra Grafi	41
5.2	Diffie-Hellman Zero-Knowledge	43
6	Bitcoin Scripting	46
6.1	Il linguaggio Bitcoin Script	48
6.1.1	Il processo di validazione: Sblocco e Blocco	48
6.1.2	Multi Signatures con Threshold	49
6.1.3	Proof-of-Burn	50
6.1.4	Timestamping	50
6.1.5	Real-life transaction con Escrow	51
6.1.6	Micro-pagamenti e Lightning Network	51
6.1.7	Freezing Funds	52
6.1.8	Pay-to-Script-Hash (P2SH)	53
7	La Blockchain	54
7.1	Il Blocco	54
7.1.1	Merkle Tree e Proof of Membership	54
7.1.2	Coinbase transaction	55
7.2	Il Network Bitcoin	56
7.2.1	Inserimento di una Transazione	57
7.2.2	La Transaction Pool e i Controlli Standard	57
7.2.3	Race Conditions e Coerenza delle Pool	58

Introduzione

Una **moneta virtuale** è, fondamentalmente, uno strumento utilizzato per effettuare transazioni all'interno di un ecosistema digitale. Per comprenderne a pieno la natura e le sfide tecnologiche, è essenziale confrontarla prima con la moneta tradizionale (reale).

Le monete reali possiedono un valore che deriva storicamente da due paradigmi principali:

- **Valore Intrinseco:** Basato sulla scarsità e sulle proprietà del materiale stesso (come nel caso dell'oro). Generare una copia di un bene con valore intrinseco ha un costo equivalente al valore del bene stesso.
- **Valore Stabilito:** Il valore della banconota è imposto e garantito da un'autorità centrale, come una Banca Centrale. In questo caso, per mantenere il valore della valuta, la sua riproduzione **falsificazione** deve essere resa artificialmente complessa, affinché il costo, la difficoltà o il rischio di creare una copia eguagli o superi il valore nominale della moneta stessa.

Passando al mondo informatico, il paradigma cambia radicalmente: fare copie di un dato digitale è un'operazione estremamente semplice e a costo quasi zero. Di conseguenza, impedire la riproduzione non autorizzata e la spesa multipla della stessa moneta virtuale rappresenta una sfida tecnologica notevole, nota in informatica come il problema del *double-spending* (o doppia spesa).

Nei sistemi di pagamento digitale tradizionali, come le carte di credito o i bonifici online, la soluzione a questo problema non risiede nella natura del dato digitale in sé. Piuttosto, il sistema si affida a **TTP (Trusted Third Parties - Enti Terzi Fiduciari)**, ovvero banche, istituti finanziari o circuiti di pagamento. Questi enti centralizzati non si limitano semplicemente a "inviare dati digitali", ma fungono da garanti assoluti: gestiscono i registri contabili, verificano le disponibilità e si occupano di tutta l'infrastruttura amministrativa e burocratica (le classiche "scartoffie") che lega e converte il dato virtuale alla valuta reale corrispondente.

È in questo contesto che si inserisce la rivoluzione del **Bitcoin**. Ideato nel 2008 sotto lo pseudonimo di **Satoshi Nakamoto**, Bitcoin nasce con il preciso obiettivo di superare i limiti strutturali legati alla necessità di fiducia e alla centralizzazione dei sistemi tradizionali.

Le sue caratteristiche fondanti, che lo distinguono dalle precedenti monete virtuali, sono:

- **Assenza di TTP:** Non esiste un'autorità centrale, un server principale o un ente terzo fiduciario incaricato di validare le transazioni o emettere nuova moneta.
- **Sistema Distribuito:** La validazione delle transazioni e la tenuta dei registri avvengono in modo decentralizzato tramite un network *peer-to-peer*, supportato da una struttura dati pubblica chiamata Blockchain.
- **Pseudo-anonimato:** Gli utenti operano e scambiano valore tramite indirizzi crittografici, senza la stretta necessità di associare un'identità anagrafica diretta per poter partecipare al network.
- **Accessibilità ed Efficienza:** Fin dalla sua concezione, e in particolare nelle sue prime fasi storiche, il protocollo ha dimostrato la possibilità di effettuare transazioni globali, incensurabili e a costi estremamente ridotti rispetto ai circuiti interbancari internazionali.

Capitolo 1

Funzioni Hash

Una funzione Hash è una funzione che prende in input una stringa di lunghezza arbitraria e ritorna in un output una stringa di lunghezza fissata che, nel nostro caso, sarà una stringa di 256 bit. Questa funzione gode di tre proprietà fondamentali:

- **Collision-Resistant:** Ossia non è possibile per avversari computazionalmente limitati trovare due input x e y tali che il $H(x) = H(y)$.
- **Hiding:** Diciamo che la funzione Hash gode della proprietà di Hiding poiché è computazionalmente intrattabile trovare a partire da $H(x)$ il valore x . Inoltre, è anche impossibile a partire da $H(x|r)$ cosa non è x , evitando quindi attacchi di brute force.
- **Puzzle-Friendly:** Questa proprietà delle funzioni hash fa' sì che noi possiamo trasformarle in una sorta di puzzle o sfida. In pratica, quello che faremo sarà fissare dei dati k e aggiungeremo una randomness x tale che $H(k|x) = y$ con y che una certo pattern. Questo pattern è, in genere, un certo numero di zeri all'inizio alla fine dell'hash e, maggiori sono gli zeri che vogliamo, maggiore sarà la difficoltà. Esattamente come un puzzle, sarà "difficile" da risolvere ma facile da verificare.

Queste proprietà sono fondamentali per implementare uno schema di **Commitment**. Possiamo immaginare il commitment come l'equivalente digitale del mettere un messaggio in una busta sigillata e lasciarla in bella vista sul tavolo: tutti possono vedere la busta (il valore è "pubblicato" e quindi non può più essere modificato), ma nessuno può verificarne il contenuto finché non decidiamo di aprirla. Formalmente, questo meccanismo si articola in due fasi, gestite da altrettanti algoritmi:

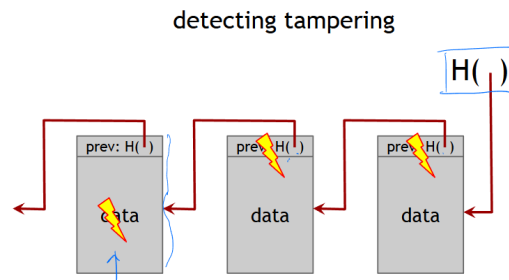
- **Fase di Commit:** L'algoritmo prende in input il messaggio m che vogliamo vincolare e un valore casuale r (necessario per garantire la proprietà di *hiding*). Restituisce in output il commitment com , ovvero la nostra "busta chiusa". Da questa stringa com è computazionalmente impossibile risalire alla coppia (m, r) .
- **Fase di Reveal e Verifica:** Quando si decide di svelare l'informazione, vengono resi pubblici m e r . A questo punto interviene l'algoritmo di *Verify*, che riceve in input i valori com , m e r . Esso calcola internamente $Commit(m, r)$ e restituisce 1 (Vero) se il risultato corrisponde esattamente al com pubblicato in precedenza, oppure 0 (Falso) altrimenti. Questo garantisce la certezza matematica che il messaggio non sia stato alterato dopo la sua pubblicazione.

1.1 Hash Pointers e Struttura della Blockchain

Da un punto di vista strutturale, una Blockchain può essere concettualmente assimilata a una *linked list* (lista concatenata). A differenza delle liste tradizionali, però, essa utilizza dei **Hash Pointers** (puntatori hash). Un hash pointer non si limita a indicare dove recuperare l'elemento precedente, ma contiene anche l'hash crittografico dei dati di quel blocco.¹

In una blockchain, ogni blocco contiene l'hash pointer al blocco **precedente**. Grazie a questo meccanismo, la catena diventa un registro a prova di manomissione *tamper-evident*: ogni blocco si fa garante dell'integrità di tutta la cronologia passata. Modificare anche un solo bit all'interno di un blocco altererebbe inevitabilmente il suo hash, rompendo la validità del puntatore nel blocco successivo e generando un effetto a cascata che invaliderebbe l'intera catena da quel punto in poi. L'unico modo per mascherare un attacco del genere sarebbe trovare una collisione nella funzione hash ma, come già discusso, per funzioni sicure ciò è computazionalmente intrattabile.

¹È importante notare che l'hash crittografico di per sé non è un indirizzo di memoria fisico o di rete; nei sistemi reali (come nei nodi Bitcoin), il client utilizza l'hash come chiave per cercare e recuperare il blocco corretto all'interno del proprio database locale.



1.1.1 Il Merkle Tree

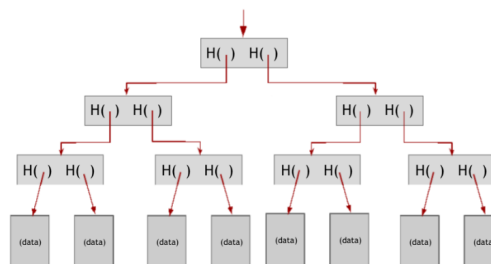
All'interno di ogni singolo blocco, le transazioni (i dati) non sono semplicemente elencate, ma sono organizzate utilizzando un'altra struttura basata sugli hash pointers: il **Merkle Tree** (Albero di Merkle).

Si tratta di un albero binario costruito con un approccio **bottom-up**:

- Le **foglie** dell'albero contengono gli hash delle singole transazioni.
- Ogni **nodo padre** viene generato concatenando gli hash dei suoi due figli e calcolandone a sua volta l'hash (es. $H(H(A) \parallel H(B))$).

Questo processo si ripete e si restringe fino a ottenere un unico hash radice, chiamato **Merkle Root**, che rappresenta crittograficamente tutte le transazioni e viene inserito nell'intestazione (header) del blocco.

Essendo costruito dal basso verso l'alto, l'albero risulta bilanciato. Questa struttura è estremamente efficiente poiché permette di verificare se una specifica transazione è inclusa in un blocco richiedendo solo il percorso dalla foglia alla radice (la cosiddetta *Merkle Proof*). Questo approccio abbate i costi computazionali e di memoria, consentendo verifiche con una complessità logaritmica ($O(\log n)$).



Verifica di inclusione: La Merkle Proof

Una delle caratteristiche più potenti del Merkle Tree è la possibilità di verificare se una specifica transazione è inclusa in un blocco senza dover scaricare

o scorrere l'intero albero. Questo meccanismo, noto come **Merkle Proof**, è alla base del funzionamento dei nodi leggeri (SPV - *Simplified Payment Verification*).

Per effettuare questa verifica, a un client sono necessari solo tre elementi:

1. L'**hash della transazione** che si vuole verificare (la foglia di partenza).
2. La **Merkle Root**, che è pubblicamente disponibile nell'header del blocco.
3. Il **Merkle Path** (o percorso di prova), ovvero la lista degli hash dei nodi "fratelli" necessari per risalire l'albero fino alla radice.

Il processo di verifica: Il client calcola l'hash della propria transazione e lo concatena con il primo hash fornito nel Merkle Path (il nodo fratello), calcolando il nuovo hash padre. Questo procedimento viene ripetuto iterativamente, salendo di livello in livello, fino a ottenere un singolo hash finale. Se l'hash calcolato dal client **coincide esattamente** con la Merkle Root del blocco, si ha la certezza crittografica che la transazione è valida e regolarmente inserita nel blocco. Poiché l'albero è binario, la quantità di hash fratelli da richiedere (il Merkle Path) cresce in modo logaritmico rispetto al numero totale di transazioni ($O(\log n)$), garantendo un'enorme efficienza in termini di banda e computazione.

1.2 Firma digitale

Una firma digitale è l'equivalente informatico di una tipica firma convenzionale. La struttura è molto simile a quella del Message Authentication, tuttavia qui abbiamo una struttura simmetrica in cui abbiamo una **chiave segreta che serve a firmare** e una **chiave pubblica che serve per verificare**. Uno schema di firma digitale è composto da tre algoritmi

1. **generateKeys:** algoritmo che genera la chiave di verifica che sarà pubblica e la chiave di firma che sarà privata. È importante notare che Bitcoin garantisce "pseudoanonimato" poiché anziché usare la propria identità, le transazioni avvengono tramite la chiave di verifica. Dunque, non abbiamo un riferimento diretto all'identità e, inoltre, a ogni persona potrebbero corrispondere infinite coppie di chiavi.
2. **sign:** è l'algoritmo che ci permette di firmare le transazioni.
3. **verify:** algoritmo che controlla che una certa firma sia valida

In particolare, lo schema di firma adottato da Bitcoin è **ECDSA** uno schema di firma particolarmente efficace che sfrutta punti di curve ellittiche per permettere una cifratura sicura con chiavi crittografiche a soli 256 bit. Per semplicità, spiegheremo lo schema DSA che è praticamente lo stesso di ECDSA ma usa spazi Z_N^* anziché curve ellittiche. Questo schema di firma si basa sul problema del **logaritmo discreto** che consiste nel problema del non riuscire a ricavare in tempo efficiente quel valore $x : g^x = X$ dati solo X e g . Gli algoritmi di firma e verifica sono dunque:

$$\begin{array}{l}
 \text{Keygen } q, G, g \\
 x \leftarrow \{1, \dots, q\} \quad h \leftarrow g^x \\
 H : \{0, 1\}^* \rightarrow \{1, \dots, q\} \\
 v_k = (G, g, q, h, H) \quad M = \{0, 1\}^* \\
 s_k = x \\
 \\
 \text{Sign}_k(m) \\
 r \leftarrow \{1, \dots, q\} \\
 R \leftarrow g^r; \quad z \leftarrow R \bmod q \\
 s \leftarrow K^{-1} (H(m) + rz) \bmod q \\
 \text{output } (R, s) \\
 \\
 \text{Ver}(v_k, m, (r, s)) \\
 w \leftarrow s^{-1} \bmod q \\
 u_1 \leftarrow g^{w H(m)} \\
 u_2 \leftarrow h^{wr} \\
 v \leftarrow u_1 \cdot u_2 \\
 \text{if } (v \bmod q = r) \text{ return 1} \\
 \text{ELSE RETURN 0}
 \end{array}$$

Come già detto tutto fa riferimento alle chiavi pubbliche che saranno le identità chiamate **indirizzi** in Bitcoin.

1.3 Basic Cryptocurrencies

Vediamo adesso una carrellata di Cryptocurrencies basilari utili a mostrare tutte le problematiche di implementare delle monete digitali.

1.3.1 GoofyCoin

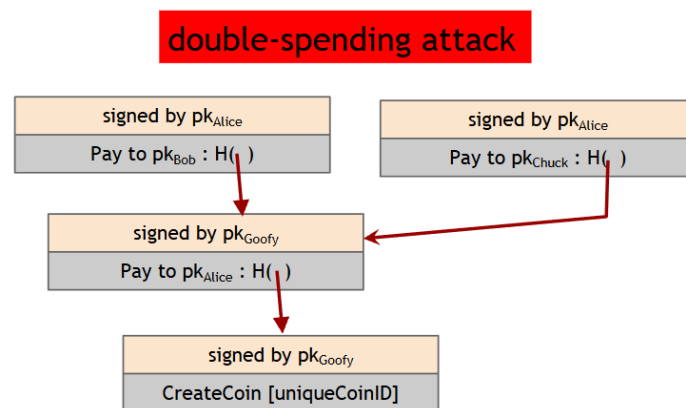
Immaginiamo questa moneta che possiede le seguenti operazioni o **transazioni**:

- **CreateCoin(uniqueCoinID)**: Questa operazione permette di creare una nuova moneta e, banalmente, colui che effettua questa transazione

la firma con la sua chiave privata. Così facendo, tutti possono verificare che quella moneta gli appartiene.

- **PayTo(indirizzo):** Questa operazione permette a qualcuno di pagare una moneta (in questo momento unitaria) a un altro indirizzo. Per farlo, oltre a fornire l'indirizzo a cui inviare la moneta, è necessario fornire l'hash pointer del blocco contenente la transazione di creazione o dell'avvenuto ricevimento della moneta.

Questo schema, seppur basilare, ci è utile a mostrare il problema del **double spending**, infatti, poiché l'unica cosa necessaria da fornire per un pagamento è un'hash pointer che provi che ci appartiene quella specifica moneta, ci basta fare tutti i pagamenti con lo stesso punto di partenza. Così facendo, riusciremo a spendere infinite volte la stessa moneta.

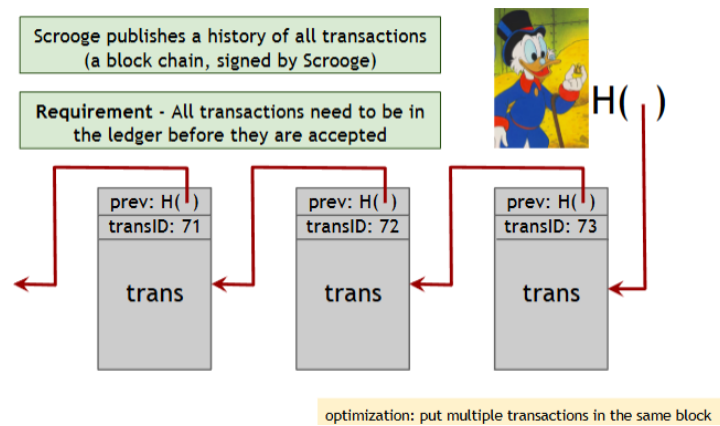


Inoltre, un altro problema è che allo stato attuale, **ognuno può creare la sua moneta gratis** e che, all'aumentare di scambi della moneta, si vanno a creare delle **catene di verifica sempre più grandi**.

Tutti questi problemi nascono dal fatto che abbiamo un sistema del tutto **decentralizzato**. Passiamo dunque alla prossima tipologia di moneta.

1.3.2 ScroogeCoin

A differenza della moneta precedente, immaginiamo di voler creare un sistema completamente centralizzato. Poiché centralizzato, questo sistema sarà **controllato da un'unica entità** che, in questo caso, sarà "Zio Paperone". Questa entità sarà l'unica a poter creare nuova moneta e a gestire il registro (cioè la Blockchain) e, quindi, decidere quali operazioni inserire e quali no.



Inoltre, anziché fare pagamenti unitari, possiamo spezzettare i pagamenti facendo sì che, ad esempio, se l'unità è 100 euro, per pagare un caffè pago 1 euro al commerciante e 99 euro a me stesso. Naturalmente ciò complica le cose perché si devono fare tante verifiche, tuttavia è fattibile.

Eliminiamo anche il problema del double spendig poiché, essendo l'entità a decidere quali operazioni accettare e quali no sul registro e, dunque, non permetterebbe mai una cosa del genere...

Tutto ciò accade se e solo se l'entità non è malevola, in caso contrario, nessun potrebbe contrastarla, rendendo inutilizzabile la moneta.

Capitolo 2

Come Bitcoin ottiene la Decentralizzazione

Uno degli obiettivi primari di Bitcoin è la creazione di un **sistema di pagamento puramente distribuito e decentralizzato**. Tuttavia, implementare e mantenere una rete senza alcun punto centrale di controllo rappresenta una sfida ingegneristica e di teoria dei giochi estremamente complessa.

Esistono molti sistemi informatici che utilizzano architetture distribuite ma non completamente decentralizzate. Un esempio quotidiano è il sistema delle email (protocollo SMTP): pur non essendoci un unico server mondiale, il sistema è "federato", ovvero si appoggia a un numero ristretto di grandi provider (es. Google, Microsoft) che fungono da intermediari per gli utenti. Bitcoin, al contrario, ambisce a eliminare del tutto queste figure centrali.

2.1 Le domande fondamentali del sistema

Per comprendere come Bitcoin realizzi questa decentralizzazione, dobbiamo capire come il protocollo risponde a cinque domande cruciali che, nei sistemi tradizionali, vengono risolte da un Ente Centrale (come una Banca o uno Stato):

1. **Gestione del registro:** Chi mantiene e aggiorna la copia ufficiale della blockchain?
2. **Validazione:** Chi possiede l'autorità per stabilire quali transazioni sono valide e quali no?
3. **Emissione:** Chi crea nuova moneta e secondo quali tempistiche?

4. **Governance:** Chi determina le regole del protocollo e come vengono implementati gli aggiornamenti software?
5. **Valore di scambio:** Come si determina il valore della moneta rispetto al mercato reale? (*A differenza delle valute fiat, il protocollo Bitcoin non gestisce questo aspetto, lasciandolo interamente alle dinamiche di libero mercato, ovvero alla legge della domanda e dell'offerta sui vari exchange*).

Oltre a questi interrogativi di sistema, vi sono questioni legate all'utente finale: come e dove conservare le proprie monete (i *wallet* crittografici) e come interfacciarsi con la rete per effettuare le operazioni.

2.2 I tre pilastri della decentralizzazione

La risposta di Bitcoin a queste sfide si articola su tre livelli principali:

- **Network Peer-to-Peer (P2P):** La rete non ha server centrali. Ogni nodo (computer) connesso è uguale agli altri, propaga le transazioni e mantiene una copia indipendente e verificata dell'intera blockchain.
- **Mining (Consenso e Creazione):** La validazione delle transazioni e la creazione di nuove monete avvengono tramite un processo competitivo noto come *Proof of Work*. Chiunque può, in linea di principio, dedicare potenza di calcolo per partecipare alla sicurezza della rete.
- **Sviluppo e Aggiornamenti:** Il codice sorgente di Bitcoin è *open-source*. Le modifiche al software vengono proposte pubblicamente e richiedono un ampio consenso da parte della rete (nodi e miner) per essere adottate.

Decentralizzazione: Teoria vs. Pratica

È fondamentale sottolineare che il sistema è aperto a tutti *sulla carta*, ma la realtà operativa attuale presenta delle sfumature. Ad esempio, il mining è teoricamente accessibile a chiunque possieda un computer; tuttavia, l'enorme competizione ha reso impossibile minare in solitaria in modo redditizio senza l'ausilio di hardware specializzato (ASIC) e senza unirsi a grandi *Mining Pool* (cooperative di miner). Questo introduce, di fatto, un certo grado di "centralizzazione pratica" in un sistema nato per essere totalmente decentralizzato.

2.3 Il Consenso Distribuito

L'obiettivo principale della rete è fare in modo che tutti i nodi raggiungano un accordo condiviso (consenso) sullo stato del registro, stabilendo, ad esempio, quale debba essere il prossimo blocco da aggiungere alla catena.

Supponiamo che l'utente A voglia inviare un pagamento all'utente B. A si preoccuperà di trasmettere (in *broadcast*) la transazione a tutti i nodi a cui è connesso. Questi la verificheranno e, se risulta valida, la manterranno in memoria pronti a inserirla in un blocco. Tuttavia, in un dato istante, non tutti i nodi hanno la stessa identica visione della rete: a causa dei fisiologici tempi di propagazione, i nodi potrebbero avere set di transazioni in sospeso leggermente diversi. Il consenso in Bitcoin, infatti, non è **immediato**, ma **emergente e a lungo termine**. I blocchi vengono proposti in media ogni 10 minuti, ma il consenso assoluto sull'irreversibilità di una transazione si consolida solo dopo circa un'ora (attendendo la conferma di almeno 6 blocchi successivi).

Le sfide del consenso: Latenza e Generali Bizantini

Immaginiamo che tre nodi (es. rosso, blu e verde) propongano contemporaneamente il proprio set di operazioni valide sotto forma di blocco. La rete deve avere un meccanismo per sceglierne uno solo, che tutti gli altri dovranno poi verificare e accettare. Questo processo è ostacolato da diversi fattori del mondo reale:

- Latenza e partizioni di rete (non tutti i nodi comunicano alla stessa velocità).
- Nodi che crashano o si disconnettono improvvisamente.
- **Nodi malevoli** che tentano di ingannare il sistema.

In informatica, la difficoltà di raggiungere un accordo in presenza di attori malevoli o fallaci è nota come il **Problema dei Generali Bizantini**.

Per risolverlo, Bitcoin assume un modello in cui i nodi sono *razionali*: essi seguono le regole del protocollo non per "bontà", ma perché il sistema di incentivi economici (la ricompensa in bitcoin) rende l'onestà molto più redditizia rispetto a un tentativo di attacco.

Il problema dell'identità: Sybil Attack

Nei sistemi distribuiti classici o chiusi, il problema dei nodi malevoli si argina legando ogni nodo a un'identità reale e verificata (es. tramite un'autorità

centrale). In Bitcoin, però, questo approccio non è applicabile: il sistema è progettato per garantire lo **pseudo-anonimato**, dove l'unica "identità" è rappresentata da una coppia di chiavi crittografiche.

Poiché chiunque può generare infinite chiavi a costo zero, la rete risulta esposta al **Sybil Attack**. In questo scenario, un singolo attaccante potrebbe creare migliaia di nodi fittizi per acquisire la maggioranza artificiale nella rete, sovrastare i nodi onesti e prendere il controllo delle decisioni.

Per neutralizzare il Sybil Attack e raggiungere il consenso senza fare affidamento su identità fisse, Bitcoin introduce un meccanismo innovativo. L'elezione del nodo che avrà il diritto di proporre il blocco successivo non avviene per votazione (dove i nodi fittizi vincerebbero), ma tramite una sorta di "lotteria" competitiva.

Algoritmo di Consenso

Ad ogni round, viene estratto in maniera randomica un leader temporaneo, che sceglie il blocco da aggiungere. Gli altri nodi della rete non possono influenzare l'estrazione, ma mantengono il potere di **validazione** e se il leader propone un blocco contenente operazioni non valide, il resto della rete lo ignorerà, rifiutandosi di estendere la blockchain a partire da quel blocco e, di fatto, creando una biforcazione. Tuttavia, poiché l'euristica per continuare una catena è quella di **continuare la catena più lunga**, quella biforcazione nata dall'inserimento di transazioni non valide morirà lì e nessuno la considererà valida. Questo meccanismo viene detto **Consenso implicito**.

Supponendo quindi di voler descrivere un algoritmo di consenso nella sua forma più semplice potremmo fare come segue:

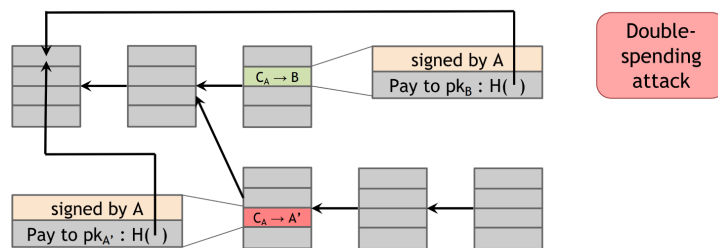
1. Le nuove transazioni vengono **inviare in broadcast** a tutta la rete.
2. Ogni nodo le mette da parte e forma dei blocchi.
3. Ad ogni round, viene scelto random un leader che propagherà il suo blocco a tutti gli altri nodi.
4. Gli altri nodi controlleranno il blocco verificando che le transazioni sono valide e, nel caso in cui lo vogliano considerare valido, faranno la prossima aggiunta inserendo in testa a quel nodo.

Analizziamo quindi cosa può e cosa non può cercare di fare un nodo malevolo:

- **Può rubare Bitcoin?** Per quanto possa provarci, se le chiavi crittografiche di un utente sono state generate in maniera sicura, questo non

potrà mai accadere poiché l'attaccante non riuscirebbe a generare una firma valida.

- **Denial of Service vs Bob?** Supponiamo che un nodo non voglia ammettere una certa transazione valida, questa potrebbe non venire mai scelta? Di fatto no poiché per quanto possa stare antipatico Bob, alcuni nodi onesti potrebbero comunque inserirla e, dunque, l'unico effetto sarebbe il delay della transazione.
- **Double-Spending?** Immaginiamo questo scenario: A fa' una transazione verso B e ottiene (senza attendere) un certo bene. La transazione con il pagamento $A \rightarrow B$ viene salvata nel prossimo blocco della blockchain, tuttavia successivamente A diventa il leader e crea una fork con un altro pagamento $A \rightarrow C$. Poiché entrambe le fork sono valide, i nodi potrebbero scegliere arbitrariamente quale allungare (di solito si procede con il principio di allungare la catena più lunga). Così facendo A ha effettivamente fatto due pagamenti spendendo, però, una sola volta monete.



Per evitare di questi problemi, è necessario che il blocco sia sufficientemente "in fondo" nella catena e, generalmente, lo consideriamo permanente dopo circa 6 blocchi successivi.

2.3.1 Incentivi e Proof of Work

Come abbiamo detto, assumeremo che tutti gli utenti della rete o quasi si comportino in maniera *razionale*. Dunque, faremo sì di dare incentivi e penalità per far sì che questi si comportino secondo le regole. Innanzitutto abbiamo due incentivi per la partecipazione attiva all'allungamento della blockchain:

1. **Block Reward:** Quando viene inserito un blocco nella catena, colui che lo ha proposto guadagna dei bitcoin tramite la *coin create transaction*. Poiché i Bitcoin hanno un valore reale di mercato fare ciò

conviene. Inoltre, scoraggiamo la creazione di blocchi non validi poiché questi non verrebbero considerati e, dunque, si "perderebbero" soldi. (parleremo tra poco di Proof of Work).

2. **Transaction Fees:** Quando viene aggiunto un blocco, oltre che un guadagno dato dalla coin create transaction, coloro che mandano le transazioni aggiungono una mancia che viene guadagnata da colui che propone il blocco. Così facendo, si incentiva a mettere transazioni con più mancia all'interno della blockchain.

Come possiamo facilmente intuire, è chiaro che il potere di aggiungere un nuovo blocco è qualcosa di molto ambito e che non possiamo effettuare tramite scelte casuali (anche perché come detto, saremmo soggetti a Sybil attack). L'idea diventa quindi non quella di scegliere un nodo del tutto a caso, bensì **scegliamo il prossimo nodo in base a quanto questo ha contribuito in termini di risorse allo sviluppo della rete**. Queste risorse devono essere non monopolizzabili e non duplicabili. Questo può essere effettuato tramite **Proof of Work** cioè quanta potenza computazionale si è data alla rete e tramite **Proof of Stake** cioè si sceglie in base a quanto è disposto a "spendere" un nodo per aggiungere il prossimo blocco. Questi sono rispettivamente adottati in Bitcoin ed Ethereum.

Hash Puzzle

Per quanto riguarda nello specifico Bitcoin, abbiamo la Proof of Work che è l'**hash puzzle**. In pratica quello che facciamo è creare il blocco includendo tutte le transazioni e l'hash pointer precedente e cercando una nonce tale che è l'intero Hash del blocco abbia una serie di zero all'inizio. Maggiori saranno gli zeri da cercare, più sarà difficile procurarsi quest'hash e, se la funzione hash è sicura, allora l'unico modo per trovare questa nonce è essere fortunati o provare tantissime combinazioni di Nonce. Come dicevamo, quindi, colui che cerca l'hash mette potenza computazionale a disposizione e fare questo ha un certo costo anche in termini di elettricità. Dunque, fare questo sforzo per inserire transazioni non valide è chiaramente uno spreco di soldi. Inoltre, l'hash puzzle gode di due proprietà principali:

1. **Difficile da calcolare:** come detto è necessaria una grande quantità di prove per trovare la giusta Nonce.
2. **Parametrizzabile:** è facile regolare la difficoltà della ricerca fixando il numero di zeri da trovare. Questo viene fatto poiché vogliamo far sì che venga aggiunto un blocco all'incirca ogni 10 minuti.

3. **Veloce da verificare:** Nonostante sia molto difficile da calcolare, resta il fatto che, una volta fornita la Nonce, questo sia estremamente facile da calcolare.

Nell'intestazione del blocco Bitcoin, lo spazio dedicato alla Nonce standard è di soli **32 bit**. Questo significa che esistono "solo" circa 4,3 miliardi di combinazioni, un numero che i moderni hardware esauriscono in frazioni di secondo senza trovare un hash valido. Per ovviare a questo problema, i miner sfruttano l'**extraNonce**, un campo inserito all'interno della transazione di ricompensa (*Coinbase Transaction*). Modificando l'**extraNonce**, cambia l'hash di quella specifica transazione, generando un effetto a cascata che modifica l'intero Merkle Tree e aggiorna la **Merkle Root** nell'header. Questa piccola modifica fornisce al miner una base di partenza completamente nuova, permettendogli di testare altri 4,3 miliardi di combinazioni di Nonce.

Sostenibilità e Difficoltà Dinamica

Perché la struttura sia sostenibile, deve garantire ai *miner* un guadagno (Block Reward + Transaction Fees) mediamente superiore ai costi operativi (hardware ed elettricità). Se questo profitto non fosse possibile, i miner spegnerebbero i loro macchinari, compromettendo la sicurezza della blockchain.

Tuttavia, il protocollo non garantisce un profitto fisso, poiché il valore della moneta fluttua sul mercato. Per compensare l'ingresso o l'uscita dei miner dalla rete, il protocollo regola automaticamente la **difficoltà dell'hash puzzle**. Questo aggiustamento non serve a salvaguardare i profitti, ma a garantire che, indipendentemente da quanta potenza computazionale sia attiva, venga scoperto un nuovo blocco in media **ogni 10 minuti**. Se molti miner entrano attratti dai profitti, la difficoltà si alza; se il prezzo crolla e molti miner spengono le macchine per non andare in perdita, la difficoltà si abbassa, permettendo alla rete di continuare a funzionare regolarmente.

L'Attacco del 51%

Come abbiamo visto, l'algoritmo di consenso funziona fintanto che la maggioranza della potenza computazionale (hash power) è in mano a nodi onesti. Ma cosa accadrebbe se un singolo attaccante (o un cartello di miner) riuscisse a controllare il **51% dell'hash power** totale della rete? Analizziamo i vari scenari:

- **Può rubare le monete di altri utenti?** No. Per autorizzare una transazione e spendere fondi altrui è necessaria la firma digitale creata

con la chiave privata del legittimo proprietario. Nessuna potenza di calcolo (salvo i futuri computer quantistici) può falsificare questa firma.

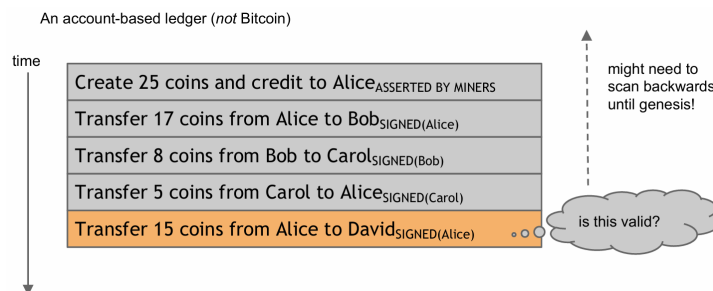
- **Può cambiare il reward del blocco (creare monete dal nulla)?**
No. Anche se il miner al 51% creasse un blocco assegnandosi 1000 BTC di ricompensa invece di quelli previsti dal protocollo, gli altri nodi (i full node che verificano le regole) scarterebbero immediatamente il blocco considerandolo non valido, indipendentemente da quanta potenza di calcolo lo abbia generato.
- **Può sopprimere (censurare) del tutto le transazioni? Nella pratica, no.** Teoricamente, se un attaccante riuscisse a mantenere la maggioranza assoluta dell'hash power per sempre, potrebbe ignorare sistematicamente i blocchi onesti contenenti la transazione sgradita, costringendo la rete ad adottare la sua catena (più lunga e priva della transazione). Tuttavia, la transazione non svanisce: rimane in attesa nella *mempool* di tutti i nodi onesti. Poiché mantenere il 51% dell'hash power comporta un costo energetico ed economico insostenibile nel lungo periodo, l'attaccante è destinato a perdere la maggioranza. Non appena ciò accade (o in momenti di forte varianza probabilistica in cui i miner onesti trovano più blocchi di fila), la rete onesta includerà la transazione. Inoltre, una censura palese e sistematica spingerebbe il *social consensus* della rete ad attuare un *Hard Fork* difensivo per neutralizzare l'hardware dell'attaccante, rendendo di fatto impossibile una censura definitiva.
- **Può distruggere la fiducia in Bitcoin (Double Spending)? Sì.** Il danno più grave di un attacco al 51% è la possibilità di effettuare *double spending* riscrivendo la storia recente della blockchain. L'attaccante può inviare dei fondi a un exchange, farsi convertire i BTC in dollari, e contemporaneamente minare in segreto una catena alternativa in cui invia quegli stessi BTC a un altro suo indirizzo. Avendo il 51%, la sua catena segreta diventerà presto la più lunga; una volta pubblicata, la rete cancellerà la transazione verso l'exchange, permettendo all'attaccante di riprendersi i fondi.

Perché non accade spesso? Oltre all'enorme difficoltà logistica e all'altissimo costo per procurarsi l'hardware e l'energia necessari per un simile attacco, vi è un fortissimo **disincentivo economico**. Se un attacco del genere avesse successo su scala visibile, la fiducia nel network crollerebbe istantaneamente. Il valore di Bitcoin precipiterebbe, rendendo di fatto inutile il bottino rubato dall'attaccante e distruggendo l'enorme investimento milionario fatto per ottenere il 51% dell'hardware.

2.4 Struttura delle Transazioni e Modello UTXO

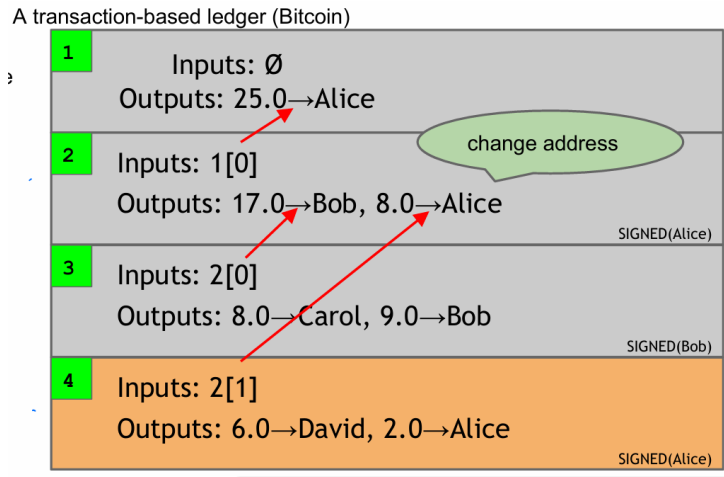
Andiamo ora ad analizzare nel dettaglio come vengono strutturate ed effettuate le transazioni all'interno della blockchain.

Uno dei problemi principali in un sistema finanziario decentralizzato è la verifica della disponibilità dei fondi: i nodi devono accertarsi che chi sta inviando del denaro ne sia effettivamente in possesso. Se adottassimo un approccio "ingenuo" (*naive ledger*), in cui il registro tiene semplicemente traccia dei trasferimenti $A \rightarrow B$, per verificare il saldo di un utente saremmo costretti a ricalcolare l'intero storico delle sue transazioni partendo dal *Genesis Block* (il primo blocco della catena). Questo causerebbe un dispendio di tempo e risorse computazionali insostenibile.



Per risolvere questo problema con la massima efficienza, Bitcoin non utilizza il concetto di "conto bancario" con un saldo aggiornabile, ma sfrutta il **Modello UTXO (Unspent Transaction Output)**.

In questo modello, le monete non esistono come saldi in un database, ma come "pezzi di metallo" digitali indivisibili (gli UTXO, appunto). Una transazione non fa altro che prendere in input dei vecchi UTXO, distruggerli, e creare in output dei nuovi UTXO.



Più nello specifico, una transazione è composta da:

- **Input:** Per giustificare e finanziare la spesa, la transazione deve fornire degli *hash pointer*. Questi puntatori indicano in modo univoco l'ID di una transazione precedente e l'indice specifico dell'output che si vuole spendere. Per poter usare questo input, l'utente deve fornire la firma digitale corrispondente (dimostrando di possedere la chiave privata di quell'indirizzo).
- **Output:** Definiscono i nuovi destinatari e i relativi importi. Poiché gli input (gli UTXO) **devono essere spesi per intero**, se il valore totale degli input è maggiore di quanto si vuole pagare al destinatario, la transazione dovrà generare un secondo output destinato al mittente stesso. Questo output aggiuntivo rappresenta il **resto** dell'operazione.

Grazie a questa struttura logica modulare, è possibile effettuare operazioni complesse con estrema facilità. Ad esempio, si possono realizzare **pagamenti congiunti** inserendo input provenienti da indirizzi diversi (per sommare i fondi); naturalmente, in questo caso, la transazione dovrà contenere una firma digitale separata e valida per ogni singolo input fornito. Inoltre, è anche possibile effettuare operazioni di **consolidamento** in cui si prendono in input più UTXO in proprio possesso per compattarli in un solo UTXO.

Capitolo 3

Consenso di Nakamoto

Prima abbiamo discusso in maniera piuttosto semplicistica come dovrebbe funzionare l'algoritmo di consenso ma, adesso, vediamo chiaramente il perché di alcune specifiche tecniche viste precedentemente.

Ricordiamo che abbiamo bisogno di:

- **Consistenza:** Cioè i nodi onesti devono poter essere d'accordo su quello che è il prossimo blocco da inserire (o quanto meno lo devono essere a lungo termine).
- **Liveness:** Cioè la blockchain deve essere in continua evoluzione.

È necessario e lo vedremo tra poco, notare che siamo in un sistema di messaggistico **asincrono** in cui i messaggi potrebbero non arrivare nello stesso ordine in cui sono stati inviati.

Analizziamo dunque il funzionamento dell'algoritmo di consenso. Come già abbiamo anticipato questo si basa sulla **Proof of Work**, una sorta di challenge in cui i miner competono per ottenere un hash di un blocco valido che abbia una certa forma dettata da vari zero all'inizio dell'hash. Ogni nodo della rete cercherà di inserire il prossimo blocco all'interno della catena più lunga. Inoltre, vogliamo che in ogni momento che tutti i blocchi a meno degli ultimi k siano considerati **definitivi**. Vogliamo dunque ottenere:

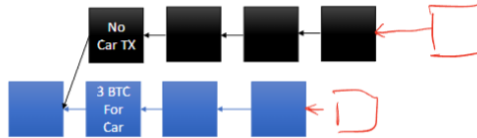
- **Consistenza:** Vogliamo che tutti i nodi onesti siano d'accordo su tutti i blocchi a meno degli ultimi k . La probabilità che il k -esimo blocco sia poi sostituito diventa $O(2^{-k})$.
- **Chain Quality:** Qualsiasi sequenza consecutiva di k blocchi contiene "un numero sufficiente di blocchi onesti". I miner che controllano una frazione p della potenza di calcolo mineranno all'incirca una frazione p dei blocchi. Cioè, se i nodi onesti possiedono l'80% dell'hash power,

allora questi dovranno poter creare all'incirca l'80% dei blocchi della catena.

- **Chain growth:** La catena cresce a un ritmo costante e prevedibile. È la proprietà di Liveness (vitalità) del sistema. Ci garantisce che la blockchain non si "congela" mai. Grazie alla calibrazione automatica della difficoltà dell'hash puzzle, la rete continuerà inesorabilmente a sfornare nuovi blocchi e a processare transazioni indipendentemente da quanti miner si disconnettono o si uniscono alla rete.

A questo punto quello che ci viene in mente è, possiamo rendere tutto il processo di mining più veloce? In effetti per come lo abbiamo descritto il sistema è molto lento e per pagare una semplice pizza dovremmo aspettare all'incirca un'ora (cioè che vengano prodotti altri 6 blocchi dopo quello che contiene la nostra transazione) e proprio questo fa' sì che la nostra moneta sia poco utilizzabile nel caso di un utilizzo in questo tipo di contesto.

Immaginiamo uno scenario in cui un attaccante cerchi di generare una catena alternativa per annullare una transazione passata, attacco noto come **Selfish Mining**. Supponiamo che il destinatario della transazione attenda z blocchi (conferme) prima di considerare il pagamento definitivo. A questo punto, l'attaccante parte con uno svantaggio di z blocchi rispetto alla catena onesta.



Definiamo le seguenti probabilità:

- p : la probabilità che il prossimo blocco venga minato dai nodi onesti.
- q : la probabilità che il prossimo blocco venga minato dall'attaccante.

Ad ogni nuovo blocco scoperto, il divario tra le due catene varia: aumenta di 1 se vince la rete onesta, diminuisce di 1 se vince l'attaccante. A vince se riesce a raggiungere il target z .

La probabilità P_z che l'attaccante riesca, prima o poi, a colmare il deficit di z blocchi e a superare la catena onesta è data dalla formula:

$$P_z = \begin{cases} 1 & \text{se } q \geq p \\ \left(\frac{q}{p}\right)^z & \text{se } p > q \end{cases}$$

Questa equazione ci fornisce due dimostrazioni cruciali sul funzionamento di Bitcoin:

1. **Vulnerabilità al 51%:** Se l'attaccante possiede la maggioranza dell'hash power ($q \geq p$), la probabilità di successo è 1 (certezza assoluta). Anche se non ci dice nulla riguardo a quanto tempo sia necessario per farlo.
2. **La regola delle 6 conferme:** Se i nodi onesti sono in maggioranza ($p > q$), la probabilità di successo dell'attaccante decade in modo esponenziale all'aumentare di z . Ad esempio, persino contro un attaccante formidabile che controlla il 30% della rete ($q = 0.3$), se si attendono $z = 6$ blocchi, la probabilità che l'attacco vada a buon fine scende a circa lo 0.6%. Per questo motivo, attendere 6 conferme rende la transazione statisticamente irreversibile.

3.1 Dimostrazione Matematica dell'attacco

Per ricavare la formula del *Nakamoto Consensus*, poniamoci l'obiettivo di trovare P_1 , ovvero la probabilità che l'attaccante recuperi uno svantaggio di esattamente 1 blocco.

Poiché il recupero di ogni singolo blocco è un evento indipendente, la probabilità di recuperare uno svantaggio di z blocchi è semplicemente:

$$P_z = (P_1)^z$$

Per calcolare P_1 , definiamo l'evento A come "l'attaccante riesce a recuperare lo svantaggio" e l'evento F come "il prossimo blocco viene minato dall'attaccante" (mentre $\neg F$ indica che viene minato dai nodi onesti). Possiamo calcolare la probabilità totale di A dividendo lo spazio degli eventi:

$$P_1 = \Pr[A] = \Pr[A \cap F] + \Pr[A \cap \neg F]$$

Applicando la definizione di probabilità condizionata, possiamo espandere la formula:

$$P_1 = \Pr[A | F] \Pr[F] + \Pr[A | \neg F] \Pr[\neg F]$$

Analizziamo i singoli termini sapendo che la probabilità di successo dell'attaccante è q e quella della rete onesta è p :

- $\Pr[F] = q$
- $\Pr[\neg F] = p$

- $\Pr[A \mid F] = 1$: se l'attaccante trova il blocco, il deficit si azzerava subito e ha recuperato con certezza.
- $\Pr[A \mid \neg F] = P_2$: se i nodi onesti trovano il blocco, il deficit passa da 1 a 2. La probabilità di recuperare da 2 è P_2 , che sappiamo essere uguale a $(P_1)^2$.

Sostituendo i valori nell'equazione otteniamo:

$$P_1 = 1 \cdot q + (P_1)^2 \cdot p$$

Riordinando i termini, arriviamo a un'equazione di secondo grado nell'incognita P_1 :

$$p(P_1)^2 - P_1 + q = 0$$

Risolvendo questa equazione (e ricordando l'identità $p + q = 1$), si ottengono direttamente due possibili soluzioni:

$$P_1 = 1 \quad \text{oppure} \quad P_1 = \frac{q}{p}$$

Poiché P_1 è una probabilità, il suo valore deve essere ≤ 1 . Se l'attaccante ha la maggioranza dell'hash power ($q \geq p$), la frazione q/p sarebbe ≥ 1 , rendendo $P_1 = 1$ l'unica soluzione logicamente accettabile (la frode avrà sicuramente successo). Se invece i nodi onesti sono in maggioranza ($p > q$), la teoria dei processi stocastici ci impone di considerare la radice strettamente minore, portandoci alla conclusione finale:

$$P_1 = \begin{cases} 1 & \text{se } q \geq p \\ \frac{q}{p} & \text{se } p > q \end{cases}$$

3.1.1 La scelta della Difficoltà e il Network Delay

Alla luce delle garanzie di sicurezza offerte dal *Nakamoto Consensus*, sorge spontanea una domanda: se il sistema è sicuro finché la maggioranza della potenza è onesta, perché imporre una difficoltà tale da generare un blocco solo ogni 10 minuti? Perché non abbassare il parametro di difficoltà per confermare le transazioni in pochi secondi?

La risposta risiede nei limiti fisici dell'infrastruttura di rete, in particolare nella **latenza di propagazione** (*Network Delay*), indicata con Δ .

In una rete P2P globale, quando un nodo trova un blocco, necessita di un tempo Δ affinché questo venga trasmesso e verificato da tutti gli altri peer. Durante questo lasso di tempo, i nodi onesti che non hanno ancora ricevuto

l'aggiornamento continuano a impiegare potenza di calcolo per risolvere un blocco ormai obsoleto. L'energia spesa in questo gap Δ è di fatto **sprecata**. Al contrario, un avversario malevolo fortemente centralizzato non subisce latenze interne, ottimizzando il 100% del suo *hash rate*.

3.1.1.1 Il Discount Rate della potenza onesta

Possiamo modellare matematicamente l'impatto di questo spreco definendo i seguenti parametri:

- n : numero totale di nodi nella rete.
- α : frazione dei nodi che si comportano onestamente (dunque vi sono in totale αn nodi onesti).
- p : probabilità che un *singolo* nodo risolva il puzzle crittografico in un dato round.

Vogliamo trovare la probabilità che la rete onesta nel suo complesso trovi un blocco in un singolo round. Per farlo, passiamo dall'evento complementare: la probabilità che **nessun** nodo onesto trovi il blocco è $(1-p)^{\alpha n}$. Dunque, la probabilità che **almeno un** nodo onesto ci riesca è:

$$1 - (1 - p)^{\alpha n}$$

Poiché in Bitcoin la probabilità p di trovare un hash valido è infinitesimamente piccola, possiamo applicare l'approssimazione binomiale (o sviluppo di Taylor al primo ordine, per cui $(1-x)^k \approx 1 - kx$ per $x \rightarrow 0$):

$$1 - (1 - p)^{\alpha n} \approx 1 - (1 - \alpha np) = \alpha np$$

Questa espressione, αnp , rappresenta la probabilità di successo per round dell'intera rete onesta. Di conseguenza, il tempo atteso (in round) per la scoperta di un nuovo blocco è semplicemente il suo inverso:

$$\frac{1}{\alpha np}$$

Poiché per ogni blocco trovato si "perdono" Δ round per la propagazione, l'efficienza reale della rete onesta è data dal rapporto tra il tempo di lavoro utile e il tempo totale impiegato (lavoro utile + spreco):

$$\text{Efficienza} = \frac{\frac{1}{\alpha np}}{\frac{1}{\alpha np} + \Delta} = \frac{1}{1 + \alpha np \Delta} \approx 1 - \alpha np \Delta$$

Questo fattore agisce come **Discount Rate** sulla potenza reale degli onesti. La potenza effettiva a disposizione dei nodi corretti non è più αn , ma diviene:

$$(1 - \alpha p n \Delta) \alpha n$$

Se la difficoltà del puzzle fosse troppo bassa, i blocchi verrebbero trovati troppo frequentemente, rendendo il parametro Δ dominante. Il *discount rate* abbatterebbe drasticamente l'efficacia dei nodi onesti, generando continue biforcazioni e avvantaggiando gli attaccanti centralizzati. Per mantenere la stabilità e la sicurezza, Satoshi Nakamoto ha calibrato la difficoltà affinché il tempo atteso per un blocco (10 minuti, ovvero 600 secondi) sia di ordini di grandezza superiore al tempo di latenza globale della rete Δ (pochi secondi), minimizzando così lo spreco di risorse.

Condizione di sicurezza

Nei sistemi distribuiti tradizionali, il consenso si basa sul conteggio dei nodi ("Un nodo, un voto") e la tolleranza viene espressa in base al numero di partecipanti malevoli (es. si tollerano al massimo 1/3 di nodi disonesti). In una rete non centralizzata come Bitcoin, questo approccio fallirebbe istantaneamente a causa del **Sybil Attack** poiché un singolo attaccante potrebbe generare milioni di nodi virtuali a costo zero, acquisendo artificialmente la maggioranza dei voti.

Per neutralizzare questo vettore di attacco, la Proof of Work sposta il paradigma di voto da "Un nodo, un voto" a "Una CPU (un hash), un voto". Di conseguenza, il protocollo Bitcoin **può tollerare un numero teoricamente infinito di nodi disonesti**, a un'unica condizione: che la somma della loro potenza computazionale (*Hash Power*) rimanga strettamente minoritaria rispetto a quella dei nodi onesti ($q < p$). Nelle analisi matematiche e statistiche del protocollo, parametri come la probabilità di successo p non rappresentano la percentuale di utenti onesti, ma la frazione di potenza di calcolo globale sotto il loro controllo.

3.1.2 Vantaggi e Svantaggi

Il sistema a consenso di Nakamoto presenta alcuni vantaggi tra cui:

- **Partecipazione anonima e facile accesso:** Poiché è possibile partecipare semplicemente con l'indirizzo pubblico della chiave generata, è chiaro che la partecipazione sia "anonima" anche se di questo parleremo dopo. Sempre per lo stesso motivo, poiché generare chiavi è estremamente semplice, è di facile accesso e, quindi, molto scalabile.

- **Impossibile prevedere il vincitore:** Se il sistema fosse prevedibile, ci dovremmo trovare ad avere a che fare con casi di collusioni in cui si cerca di corrompere il leader per fargli inserire o non inserire certe transazioni. Invece, poiché il vincitore non è prevedibile, questo non è chiaramente uno scenario fattibile.
- **Tolleranza di tanti nodi malevoli:** Per quanto detto prima, fintanto che la potenza di calcolo dei nodi malevoli è inferiore rispetto alla potenza di calcolo dei nodi onesti, vi è una grossa tolleranza.

Come tutte le cose belle, però, abbiamo alcuni svantaggi parecchio ovvi che riguardano principalmente il fatto che questo metodo è **lento** e, quindi, non permette pagamenti immediati e, a causa delle proof-of-work, abbiamo un **grosso dispendio energetico** rendendo, di fatto, il consumo di energia tra i più alti rispetto perfino ad alcuni paesi del mondo.

Capitolo 4

Anonimato e Privacy

Prima di analizzare le tecniche di anonimato all'interno della blockchain, è necessario definire con precisione cosa significhi questo termine in ambito informatico. Generalmente, il concetto si presta a due interpretazioni principali:

- Interagire **senza utilizzare il proprio vero nome** (ma impiegando un alias testuale o numerico).
- Interagire **senza rivelare alcuna identità** o traccia riconducibile al singolo individuo.

Nel caso di Bitcoin, la prima interpretazione è chiaramente soddisfatta, poiché gli utenti utilizzano gli hash delle proprie chiavi pubbliche (gli indirizzi) come identità. Tuttavia, la seconda interpretazione non lo è. Questa distinzione ci porta a definire il sistema Bitcoin non come anonimo, bensì come **pseudo-anonimo**.

Lo pseudo-anonimato, da solo, non è sufficiente a garantire la privacy su un registro pubblico. In crittografia, il vero anonimato si definisce come la somma di due proprietà: **Pseudo-anonimato + Unlinkability** (Non-collegabilità). L'*unlinkability* è la proprietà per cui differenti interazioni dello stesso utente non possono essere correlate tra loro dal sistema o da un osservatore esterno. Per ottenere un'effettiva privacy in Bitcoin, dovrebbero verificarsi tre condizioni di *unlinkability*:

- Deve essere computazionalmente difficile ricollegare indirizzi diversi allo stesso utente reale.
- Deve essere difficile ricollegare transazioni diverse al medesimo utente.
- Deve essere difficile ricollegare il mittente di un pagamento al suo effettivo destinatario.

Mentre per i primi due casi è possibile adottare euristiche e buone pratiche (come generare un nuovo indirizzo per ogni transazione), il terzo caso è intrinsecamente violato dall'architettura base del protocollo: analizzando una singola transazione standard, il collegamento logico tra l'input (mittente) e l'output (destinatario) è pubblico e palese. Offuscare questo legame diventa fattibile solo se consideriamo la transazione non come un singolo evento isolato, ma come un flusso all'interno di un grafo di transazioni complesse.

Poiché garantire una completa *unlinkability* sull'intero network è impossibile, l'obiettivo crittografico è quello di rendere anonima almeno una porzione delle transazioni, nascondendole all'interno di un **Anonymity Set** (Insieme di Anonimato). L'efficacia di queste tecniche dipenderà direttamente dal *Threat Model*: dobbiamo cioè stabilire a priori **chi è l'attaccante e quali sono le sue capacità** di analisi della rete.

4.1 Perché vogliamo l'anonimato?

Poiché la blockchain è un *ledger* (registro) completamente pubblico, l'assenza di anonimato comporta enormi criticità. Nel sistema bancario tradizionale, il registro delle transazioni è privato e custodito da un ente fiduciario; nessuno, tranne le autorità competenti, può accedere allo storico finanziario di un cittadino. Al contrario, in Bitcoin chiunque può ispezionare il registro, calcolare il saldo di una certa identità pseudo-anonima, e tracciarne ogni singolo movimento in entrata e in uscita.

Questa trasparenza estrema espone gli utenti a veri e propri tracciamenti finanziari. Se l'identità reale di un utente viene associata al suo indirizzo, è possibile per chiunque compiere azioni di spionaggio (ad esempio, un'azienda potrebbe monitorare i fornitori e i volumi d'affari di un competitor).

Spesso si associa erroneamente la richiesta di anonimato finanziario ad attività illecite (acquisto di beni illegali, riciclaggio, evasione fiscale). Sebbene la tecnologia possa essere abusata, il diritto alla privacy finanziaria è un'esigenza legittima per proteggersi da sorveglianza di massa e attacchi mirati. Inoltre, anche i fondi di provenienza illecita, per poter essere utilizzati nell'economia reale, prima o poi devono transitare attraverso un punto di conversione in valuta *fiat* (*exchange* con procedure KYC/AML), rendendo di fatto possibile il tracciamento da parte delle forze dell'ordine. La neutralità della tecnologia crittografica è simile a quella della rete Tor: uno strumento *agnostico* che fornisce privacy a prescindere dall'intento etico o meno dell'utente finale.

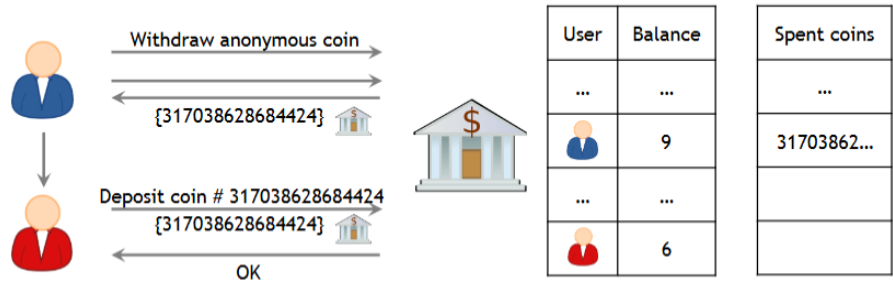
4.1.1 Anonymous e-cash e Blind Signatures

Una delle primissime e più eleganti soluzioni per i pagamenti digitali anonimi fu proposta dal crittografo David Chaum nel 1982. L'idea alla base di *e-cash* era quella di consentire pagamenti digitali mantenendo una banca come autorità centrale fidata per la contabilità, ma impedendole matematicamente di tracciare le abitudini di spesa dei propri clienti.

Il protocollo si basa sul concetto crittografico delle **Blind Signatures** (Firme Cieche) e si articola in tre fasi:

- **1. Fase di Prelievo (*Withdrawal*):** L'utente Alice genera un numero seriale casuale S , che rappresenterà univocamente la moneta elettronica. Per nascondere alla banca, Alice applica una funzione di *blinding* (mascheramento) utilizzando un fattore casuale r , ottenendo un *commitment* $M = f(S, r)$. Alice invia M alla banca richiedendo un prelievo. La banca autentica Alice, verifica i fondi sul suo conto e li detrae. Successivamente, la banca applica la propria firma digitale sul *commitment* cieco, generando $\text{Sign}_{\text{bank}}(M)$, e la restituisce ad Alice. La banca ha firmato la moneta senza averne mai visto il vero numero seriale S .
- **2. Fase di Unblinding:** Ricevuta la firma cieca, Alice utilizza il fattore r per rimuovere il mascheramento matematico applicato in precedenza. Grazie alle proprietà omomorfe delle *blind signatures*, questa operazione restituisce una firma della banca perfettamente valida applicata direttamente al seriale in chiaro: $\text{Sign}_{\text{bank}}(S)$.
- **3. Fase di Deposito (*Deposit*):** Alice desidera acquistare un bene da Bob e gli consegna la moneta $(S, \text{Sign}_{\text{bank}}(S))$. Bob verifica criticograficamente la firma usando la chiave pubblica della banca e, per concludere la transazione, invia la moneta alla banca per farsi accreditare l'importo. La banca riceve il seriale S , verifica la propria firma e controlla nel suo database che S non sia mai stato speso prima (prevenendo il *Double Spending*). Se S è nuovo, la banca accredita i fondi sul conto di Bob.

La garanzia di Anonimato: Durante il deposito, la banca vede il numero seriale S per la prima volta. Non ha alcun modo crittografico per collegare questo seriale S al *commitment* M che aveva firmato ad Alice durante la fase di prelievo. Di conseguenza, pur mantenendo il controllo assoluto contro la doppia spesa, l'autorità centrale non può assolutamente sapere chi ha pagato Bob, garantendo così un anonimato totale e matematicamente provato per il pagatore.



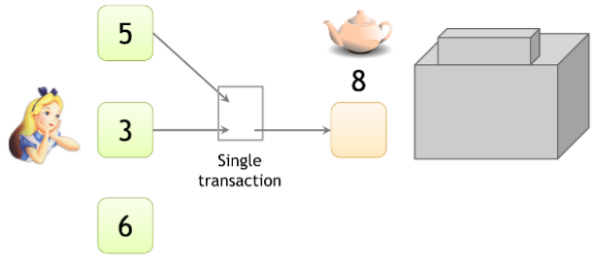
Naturalmente, per garantire tale anonimato abbiamo bisogno di spese unitarie o, al più, un numero limitato di quantità "prestabilite" in maniera tale che la banca non possa fare linking mediante la cifra spesa. Si presuppone chiaramente che il sistema sia utilizzato da più persone poiché altrimenti il link risulta banale.

Il vero e proprio problema è il fatto che i protocolli interattivi con la banca sono complicati da gestire e da decentralizzare soprattutto.

4.1.2 Anonimato grazie a nuovi indirizzi

Supponiamo che A voglia ottenere il massimo anonimato dal sistema block-chain e che, intuitivamente, decide di usare sempre indirizzi diversi per ogni transazione.

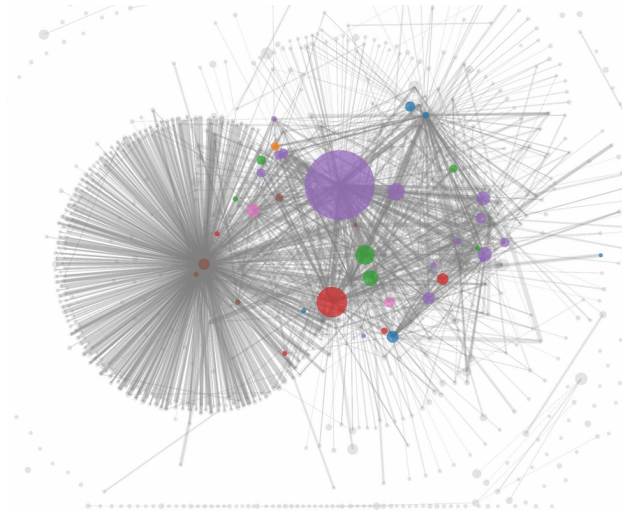
Siamo adesso nella situazione in cui A voglia comprare un oggetto e, per farlo, necessita di un certo numero di monete che sono, quindi, dentro tanti UTXO diversi che hanno una firma diversa.



Poiché un attaccante vede una transazione verso una certa autorità,¹ questo vede prima la transazione di compattazione degli uTXO e, quindi, riesce a collegare che quei due UTXO appartenevano alla stessa entità. Non solo, se il pagamento non è per intero, allora è necessario dare il resto che, quindi, apparterrà alla stessa entità del pagamento.

¹Vedremo dopo come questa cosa venga capita

Tramite queste intuizioni, possiamo tracciare un grafo di questo tipo:



Possiamo notare che quelli che ricevono tante transazioni sono, in genere, le autorità di cui parlavamo prima. Tra queste, possiamo trovare le più note **silkroad**, **satoshi dice**, ecc.. Siamo a tutti gli effetti riusciti a capire come individuare le grossi autorità ma non è ancora ben chiaro come possiamo a tutti gli effetti deanonimizzare i singoli utenti. Per farlo, l'approccio più semplice è quello del *social engineering*:

- **Transazioni dirette:** Se io faccio in modo che qualcuno mi paghi, trovo il suo indirizzo e, tramite il blockchain explorer, posso vedere quanto si aveva nel portafoglio, da chi ha ricevuto questi soldi ecc.
- **Fornitori di Servizi:** Poiché non tutti possono minare Bitcoin, la maggior parte li compra da servizi esterni come binance e quei servizi devono per leggere conoscere l'identità della persona.
- **Careless:** La gente può deanonimizzarsi da sola pubblicando l'indirizzo online per ricevere donazioni ecc.
- **Attacchi retroattivi:** Poiché vengono scoperte algoritmi sempre più efficaci per raggruppare indirizzi e dato che la blockchain è immutabile, è possibile che in futuro venga scoperto un modo ancora più forte per deanonimizzare qualcuno.

4.2 De-anonimizzazione a livello Network

Un altro problema è che pur evitando di pubblicare il proprio indirizzo online, la comunicazione a livello di rete potrebbe fare in modo che qualcuno riesca a deanonimizzarci attraverso l'indirizzo IP.

Una regola generale è *"Il primo nodo che informa qualcuno di una transazione, è probabile che sia la sua fonte"*.

4.2.1 Rete Tor

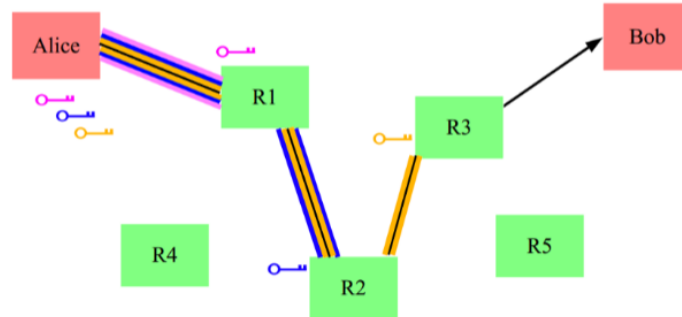
Per evitare questo problema, la soluzione è l'uso della rete **Tor** un protocollo di rete che si pone come obiettivo quello di garantire l'anonimato a livello di rete.

Quello che vogliamo è una comunicazione di questo tipo:

- **Senders:** Questi invieranno i messaggi all'interno della rete di comunicazione.
- **Communication Network:** Qui arriveranno i messaggi e lì non è ben chiaro il mittente e il destinatario. Questo a patto che questo sistema sia usato da un certo numero di persone poiché altrimenti sarebbe banale il collegamento.
- **Recipients:** Questi riceveranno i messaggi attraverso la rete in maniera del tutto anonima. Cioè, il recipients non saprà nemmeno chi gli ha inviato il messaggio. Per quanto possa sembrare assurdo, questo è appropriato dato che Tor è progettato per la navigazione sul web.

Per fare questo, Tor opera secondo un modello di **cifratura a strati**. Questo sistema è considerato sicuro se **almeno uno dei router è onesto**.

Quando Alice deve comunicare con Bob utilizzerà delle chiavi di cifratura valide per la sessione corrente per **cifrare un messaggio n volte, dove n è il numero di macchine intermedie per cui passa il messaggio**. Ogni macchina intermedia applica una chiave di cifratura per togliere uno strato di cifratura al messaggio fino all'ultima macchina del path prestabilito da Alice. A quel punto il messaggio, ormai in chiaro passa dall'ultima macchina ad Alice.



Questo sistema è sicuro nel caso in cui ci sia almeno uno nodo onesto. Guardiamo i vari casi:

- **Se R2 e R3 sono malevoli:** R1 è onesto e sa da chi viene il messaggio. Tuttavia, poiché onesto questa informazione non verrà trasmessa e, dunque, R2 e R3 possono conoscere B ma non A.
- **Se R1 e R3 sono malevoli:** In questo caso, R2 riceverà l'identità di A inoltrata da R1, tuttavia poiché onesto ignorerà tutte le identità inoltrate e garantirà l'anonimato di A.
- **Se R1 e R2 sono malevoli:** In quest'ultimo caso R3 non invierà comunque le identità ricevute da R2 garantendo l'anonimato di A.

Statisticamente, più server ci sono più il sistema è probabile che sia sicuro poiché è più probabile che ci siano server router onesti. Al tempo stesso, più server ci sono maggiore è la latenza.

4.3 Mixing e Anonimato nelle Criptovalute

Un'idea utile per ottenere anonimato nelle criptovalute è quello di usare degli intermediari che fanno da **mixing network**.

Questo può essere utilizzato per ottenere anonimato nelle transazioni di criptovalute operando come meccanismo di dissociazione tra indirizzi di input e indirizzi di output per evitare il linking visto precedentemente. L'idea è quella di avere un intermediario per noi a cui mandiamo i messaggi e che poi li manda per noi attraverso dei **chunk**.

4.3.1 Online Wallet

I wallet online rappresentano una prima forma di mixing, sebbene questo non sia generalmente il loro scopo primario. Questi servizi gestiscono collettivamente i fondi di numerosi utenti, facilitando implicitamente un certo grado di commistione tra i depositi.

Quando un utente trasferisce Bitcoin a un wallet online e successivamente li preleva, tipicamente riceve unità di valuta diverse da quelle originariamente depositate. Questo processo introduce un elemento di dissociazione che rende più complessa l'analisi della provenienza dei fondi.

Questi, però, presentano diverse limitazioni:

1. **Assenza di garanzie:** La maggior parte dei wallet online non si impegna formalmente a implementare pratiche di mixing robuste. L'effetto di mixing deriva principalmente da procedure operative interne finalizzate all'ottimizzazione della gestione dei fondi, piuttosto che da un impegno esplicito verso la tutela della privacy degli utenti.
2. **Conservazione dei registri:** Anche ipotizzando un efficace processo di mixing interno, i wallet online generalmente mantengono registri dettagliati delle transazioni effettuate dagli utenti. Questa pratica è adottata sia per ragioni di convenienza operativa, facilitando la risoluzione di controversie e la gestione del servizio, sia per ottemperare a requisiti normativi sempre più stringenti in materia di antiriciclaggio e contrasto al finanziamento del terrorismo.
3. **Requisiti di Identificazione:** I wallet online sono sottoposti a regolamentazioni e richiedono la verifica degli utenti.

Quello che vorremmo in realtà sono due caratteristiche fondamentali:

- **Impegno a non conservare registri:** I servizi di mixing dedicati dichiarano esplicitamente di non mantenere archivi delle transazioni processate, eliminando teoricamente la possibilità di ricostruire i collegamenti tra indirizzi di input e output.
- **Assenza di requisiti di Identificazione:** A differenza dei wallet regolamentati, i mixing service non richiedono la verifica dell'identità degli utenti, preservando l'anonimato anche rispetto al provider del servizio stesso

4.3.2 Principi di Mixing

Sono stati ideati dei principi fondamentali per ottenere anonimato attraverso sistemi di mixing:

1. **Serie di Mixer:** L'idea è quella di implementare una sequenza di operazioni di mixing successive piuttosto che affidarsi a un singolo intermediario. Questo ha diversi vantaggi:
 - **Riduzione del rischio di compromissione:** L'utilizzo di molteplici mix riduce drasticamente la vulnerabilità del sistema poiché per eliminare l'anonimato bisognerebbe compromettere tutti i mixer contemporaneamente.
 - **Incremento dell'Anonymity set:** Ogni passaggio attraverso un mix diverso espande l'insieme delle transazioni con cui i fondi dell'utente vengono mescolati, aumentando progressivamente la complessità dell'analisi necessaria per deanonimizzare le transazioni.
2. **Uniformità delle Transazioni:** Per evitare il link delle transazioni attraverso l'analisi degli importi, è opportuno che i servizi di mixing paghino attraverso transazioni con pagamenti standard detti **chunk**. Così facendo, poiché tutti spendono/ricevono la stessa quantità di monete in una transazione, non è possibile ricollegare l'origine della moneta alla sua destinazione. Vanno bene anche **diverse taglie di chunk**.
3. **Automatizzazione Lato Client:** Per evitare problemi legati all'elevata complessità della gestione di questo tipo di sistema e, dunque, il possibile errore umano, avrebbe senso far sì che sia lo stesso wallet a gestire questo tipo di meccanismi lato Client in maniera del tutto nascosta al client.
4. **Commissioni con All-or-Nothing:** Poiché questo è un servizio di un qualche tipo, è necessario chiaramente pagarlo. Dunque, per garantire l'anonimato anche del servizio stesso, è necessario adottare un meccanismo **All-or-Nothing** in cui la commissione non è una percentuale della transazione stessa, bensì l'intero importo della transazione che va al servizio con una probabilità ad esempio del 0,01%. Questo preserverebbe l'uniformità delle transazioni e proteggerebbe l'anonimato del servizio stesso.

Allo stato attuale delle cose, nessuno adotta realmente questo tipo di mixing per vari motivi tra cui la legge stessa ma anche perché, volendo i servizi

lavorare in anonimo, è chiaro che non sarebbe possibile imputarli in alcun modo e, dunque, nessuno avrebbe fiducia in un sistema del genere.

4.4 Coinjoin

Poiché i sistemi di mixing centralizzati hanno dei vincoli imposti dalla centralizzazione stessa, si è cercato di passare a un sistema di **mixing decentralizzato** così che non ci sia la necessità di un ente a cui affidarsi e che sia l'utente stesso a gestire i suoi fondi all'interno del processo di mixing.

Qui nasce **CoinJoin** proposto da *Greg Maxwell*. Questo è un protocollo di mixing decentralizzato che sfrutta una proprietà fondamentale delle transazioni di Bitcoin ossia il sistema di **firme aggregate** in un'unica transazione. Vedremo nei capitoli successivi i dettagli di questa implementazione. Per fare capire come funziona, l'idea è che per firmare una transazione facciamo in modo che la firma sia generata da tutti coloro che partecipano a questo meccanismo generando firme indipendenti che singolarmente non sono valide ma che aggregate assieme lo sono.

L'algoritmo procede come segue:

1. **Individuazione dei Peer:** I partecipanti interessati a un round di mixing per un importo specifico devono trovarsi. Comunemente, ciò avviene tramite un server di coordinamento a cui ci si connette, spesso via Tor, per preservare l'anonimato dell'indirizzo IP. I partecipanti si registrano per un round specifico, indicando l'intenzione di mixare un determinato importo, senza ancora rivelare gli UTXO specifici.
2. **Fornitura Input/Output:** Ogni utente invia al coordinatore l'UTXO che vuole mixare e fornisce una prova di possesso (firma). In questa fase, il coordinatore sa che l'indirizzo *A* appartiene all'utente 1. L'utente deve fornire l'indirizzo di "uscita" dove riceverà i fondi puliti. Per evitare che il coordinatore colleghi l'input all'output, l'utente invia l'indirizzo di output tramite una nuova connessione Tor e utilizza una firma cieca.
3. **Costruzione della Firma:** Il coordinatore invia la transazione completa a tutti i partecipanti. Ogni utente controlla la transazione. Firma solo se vede che tra gli output è presente il suo indirizzo con la cifra corretta. Se il coordinatore provasse a rubare i soldi o a cambiare un indirizzo, l'utente non firmerebbe e la transazione fallirebbe. Non c'è rischio di furto.

4. **Verifica della Firma:** Quando il coordinatore raccoglie le firme di tutti i partecipanti (o della soglia minima richiesta), la transazione diventa valida secondo le regole del protocollo Bitcoin. Il coordinatore verifica che tutte le firme siano valide e che la transazione non violi alcuna regola. La transazione viene inviata alla rete Bitcoin. Sulla blockchain apparirà un unico evento in cui 100 persone hanno mescolato i loro soldi, rendendo impossibile per un osservatore esterno sapere chi ha ricevuto cosa.

Questo sistema presenta però alcuni problemi:

- **Individuazione dei Peer:** Un primo ostacolo significativo consiste nell'identificazione di altri utenti interessati al mixing, mantenendo contemporaneamente l'anonimato.
- **Denial of Service:** Partecipanti malevoli potrebbero compromettere l'efficacia del protocollo rifiutandosi di firmare la transazione nelle fasi finali, causando un fallimento del processo di mixing dopo che gli altri utenti hanno già investito tempo e risorse.

Per quanto riguarda l'individuazione dei Peer abbiamo già parlato di come le rete Tor sia una soluzione a questo problema creando delle lobby in cui effettuare la ricerca. Per quanto riguarda, invece, il Denial of Service, possiamo applicare lo stesso meccanismo che adottiamo su Bitcoin cioè il **proof of work** o il **proof of burn**.

Capitolo 5

Zero Knowledge Proofs

Un tipico scenario in crittografia è quello in cui due parti che non si fidano l'un l'altra devono rivelare qualche informazione segreta in maniera sicura **senza che queste rivelino ulteriori informazioni che non vorremmo diffondere**.

Nello specifico, nel nostro caso vogliamo dimostrare di possedere determinati requisiti senza rivelare troppe informazioni.

Definiamo una **dimostrazione** come un qualcosa che convince della validità di un certo asserto. In matematica questa cosa è intesa in senso statico ma per il nostro caso sarà un concetto dinamico effettuato attraverso un sistema interattivo.

Distingueremo tra Prover P cioè colui che vuole dimostrare la validità dell'asserto e verifier V ha il compito di verificare la prova fornita dal prover.

In genere, è più facile verificare che dimostrare o meglio provare e, questo concetto, è rappresentato perfettamente dai problemi NP. Questi sono dei particolari problemi la cui risoluzione è complessa ma che, una volta fornita la soluzione, questa è facile da verificare.

Poiché vogliamo che non venga rivelata alcuna informazione in termini di conoscenza, è necessario definire cosa si intende per conoscenza. In un tipico scenario di comunicazione tra A e B , diremo che B ha un **guadagno in termini di conoscenza se la sua capacità computazionale è arricchita dalla conversazione**.

5.1 Prove Interattive

Come già accennato, effettueremo dimostrazioni secondo prove interattive. Queste rispecchiano due proprietà specifiche:

- **Completeness:** P dovrebbe essere in grado di convincere V della validità di asserti veri.

$$\forall x \in L \quad \Pr[\langle P, V \rangle(x) = 1] \geq 2/3$$

- **Soundness:** Un P disonesto non dovrebbe essere in grado di convincere V della validità di un asserto falso.

$$\forall x \notin L, \forall B \quad \Pr[\langle B, V \rangle(x) = 1] \leq 1/3$$

Informalmente, un sistema di prove interattive per un linguaggio L è detto **zero knowledge** se qualsiasi cosa è calcolabile interagendo con P su input $x \in L$ è calcolabile a partire da x soltanto.

Cioè, il Verificatore (V) non impara nulla di nuovo dall'interazione con il Prover (P) perché, se avesse voluto, avrebbe potuto simulare l'intera conversazione da solo, senza parlare con nessuno.

Supponiamo che un'entità A possa sapere con certezza il risultato di una determinata operazione e che un'entità B voglia verificare questa sua abilità. Dunque B sceglie a caso un'operazione e poi A darà la sua risposta. Se indovina, potrebbe avere avuto fortuna e lo potremmo accettare o fare di nuovo il test, altrimenti lo rigettiamo. Via via che facciamo altre prove, la probabilità che lui abbia solo avuto fortuna va scemando.

Sia *view* la variabile aleatoria che denota ciò che vede V durante l'esecuzione di $(P, V)(x)$ che, nel nostro caso, saranno i messaggi scambiati tra P e V. Diciamo che (P, V) è un interactive proof system che fornisce **Perfect Zero Knowledge** per il linguaggio L se $\exists M \text{ PPTM} : \forall x \in L, a > 0, \forall h : |h| < |x|^a \quad M(x, h)^1$ sono distribuite in modo identico.

Cioè quello che riesce a produrre la macchina a stati finiti M è del tutto indistinguibile dall'esecuzione reale del protocollo.

5.1.1 Isomorfismo tra Grafi

Due grafi si definiscono **isomorfi** se esiste un'applicazione biunivoca tale che ogni nodo di G_1 può essere mappato in un nodo di G_2 preservando il grado.

Questo problema non è risolvibile tramite un algoritmo polinomiale e questo lo rende perfetto per il nostro scopo di avere una **Zero Knowledge proof**.

Costruiamo il nostro schema come segue:

¹h rappresenta la conoscenza pregressa di un verificatore malevolo che vuole rubare delle informazioni. Mentre x rappresenta l'input comune tra prover e verifier.

- Il Prover possiede due grafi, G_1 e G_2 , che sono isomorfi. Conosce cioè una funzione (una permutazione ρ) tale che $G_2 = \rho(G_1)$. Questo ρ è il **segreto** che il Prover non vuole rivelare.
 - Il Prover sceglie una permutazione casuale π :
 - Crea un nuovo grafo H , che è l'immagine di G_1 tramite π ($H = \pi(G_1)$).
 - Invia il grafo H al Verifier.
- Poiché H è "coperto" dalla casualità di π il Verifier non può capire come H sia collegato a G_1 o G_2 solo guardandolo.
- Il Verifier invia una sfida casuale c . Può scegliere solo tra due valori: 1 o 2.
 - Se $c = 1$, sta chiedendo: "Dimostrami che H è isomorfo a G_1 ".
 - Se $c = 2$, sta chiedendo: "Dimostrami che H è isomorfo a G_2 ".
 - Il Prover calcola la risposta σ in base alla sfida:
 - Se $c = 1$ Il Prover rivela $\sigma = \pi$. Infatti, H era stato costruito proprio come $\pi(G_1)$.
 - Se $c = 2$: Il Prover deve mostrare la relazione tra G_2 e H . Poiché conosce sia il segreto ρ (che collega G_1 a G_2) sia π (che collega G_1 a H), può calcolare la composizione $\sigma = \pi \circ \rho^{-1}$.
 - In fine, il Verifier riceve σ e controlla se $H = \sigma(G_c)$ se l'uguaglianza è verificata, se è così, allora il prover passa il turno, altrimenti viene rigettato.

Dimostriamo le proprietà

- **Completeness:** La proprietà di completezza è verificata banalmente. Se il Prover è onesto e possiede effettivamente l'isomorfismo ρ tale che $G_2 = \rho(G_1)$, allora:
 - Se $c = 1$, può sempre fornire $\sigma = \pi$, poiché per costruzione $H = \pi(G_1)$.
 - Se $c = 2$, può sempre fornire $\sigma = \pi \circ \rho^{-1}$. Infatti:

$$\sigma(G_2) = (\pi \circ \rho^{-1})(G_2) = \pi(\rho^{-1}(G_2)) = \pi(G_1) = H$$

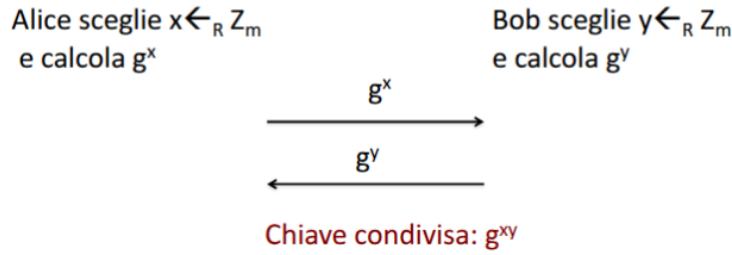
In entrambi i casi, il Verifier accetterà la prova con probabilità 1.

- **Soundness:** La soundness garantisce che un Prover disonesto non possa ingannare il Verifier. Se G_1 e G_2 non sono isomorfi, il Prover non può generare un grafo H che sia contemporaneamente isomorfo a entrambi. Egli deve quindi "scommettere" sulla sfida del Verifier: Può generare H isomorfo a G_1 (sperando che esca $c = 1$). Può generare H isomorfo a G_2 (sperando che esca $c = 2$). In ogni round, la probabilità di successo dell'attaccante è esattamente $1/2$. Iterando il protocollo per n round indipendenti, la probabilità che un baro riesca a convincere il Verifier scende esponenzialmente a $(1/2)^n$. Per $n = 100$, tale probabilità è talmente piccola da essere considerata trascurabile.
- **Perfect Zero Knowledge (PZK):** dobbiamo dimostrare che il Verifier non riceve alcuna informazione sul segreto ρ . Costruiamo una macchina M (il Simulatore) che non conosce ρ ma che è in grado di produrre una trascrizione (una *view*) indistinguibile da quella reale. Il Simulatore M opera nel seguente modo:
 1. Sceglie a caso una sfida "prevista" $c^* \in \{1, 2\}$.
 2. Sceglie una permutazione casuale σ^* .
 3. Calcola il grafo $H^* = \sigma^*(G_{c^*})$.
 4. Produce la tripla (H^*, c^*, σ^*) .

Notiamo che nella realtà il Verifier riceve prima H , poi sceglie c , e infine riceve σ . Il Simulatore, invece, costruisce la risposta a ritroso: sceglie prima la sfida e poi "aggiusta" il grafo H di conseguenza. Poiché in un protocollo PZK le distribuzioni sono identiche, un osservatore che guarda il log della comunicazione non potrà distinguere se H sia stato inviato prima o dopo la scelta di c . Dato che M è riuscito a generare una *view* valida senza conoscere ρ , concludiamo che la *view* reale non contiene alcuna traccia di ρ . Il guadagno di conoscenza del Verifier è quindi pari a zero.

5.2 Diffie-Hellman Zero-Knowledge

Parliamo adesso di un altro protocollo, il protocollo **Diffie-Hellman**. Siano G, g, m i parametri pubblici:



Questo rappresenta un protocollo standard di distribuzione di chiave crittografica simmetrica. Nel nostro caso vorremo provare che (g, g^x, g^y, g^{xy}) è una **quadrupla Diffie-Hellman** valida, cioè che:

$$DL_g(g^y) = DL_{g^x}(g^{xy})$$

Vediamo come funziona la prova Zero Knowledge. Siano $g^x = X, g^y = Y, g^{xy} = W$:

1. Il Prover sceglierà un valore random $r \xleftarrow{R} \{1, \dots, m\}$ e porrà $u \leftarrow g^r, v \leftarrow X^r$ inviandoli al verifier.
2. Il verifier sceglierà $c \xleftarrow{R} \{1, \dots, m\}$ e lo invierà al prover.
3. Il Prover calcolerà $z = r + cy \pmod m$ e lo invierà al verifier.
4. A questo punto il verifier verificherà se:
 - $g^z = u \cdot Y^c$
 - $X^z = v \cdot W^c$

Dimostriamo le proprietà

- **Completeness:** Se la quadrupla è valida, allora avremo che:

$$\begin{aligned} - g^z &= g^{r+cy} = g^r \cdot (g^y)^c. \\ - X^z &= (g^x)^{r+cy} = g^{xr} \cdot (g^{xy})^c = v \cdot W^c. \end{aligned}$$

- **Soundness:** Supponiamo che un attaccante voglia convincere il Verifier che (g, X, Y, W) sia una quadrupla di Diffie-Hellman valida, quando in realtà non lo è. Poniamo quindi $W = g^s$ con $s \neq xy$. Per superare il controllo, l'attaccante deve scegliere i valori di impegno u e v . Siano questi $u = g^\alpha$ e $v = g^\beta$ per qualche α, β scelti dall'attaccante. Al passo 4, il Verifier accetterà la prova solo se l'attaccante è in grado di fornire un valore z che soddisfi contemporaneamente le due equazioni di verifica:

1. $g^z = u \cdot Y^c \implies g^z = g^\alpha \cdot g^{yc} = g^{\alpha+yc}$
2. $X^z = v \cdot W^c \implies (g^x)^z = g^\beta \cdot (g^s)^c \implies g^{xz} = g^{\beta+sc}$

Dalla prima equazione ricaviamo che deve essere $z \equiv \alpha + yc \pmod{m}$. Sostituendo questo valore di z nella seconda equazione (espressa in termini di esponenti), otteniamo:

$$\begin{aligned} x(\alpha + yc) &\equiv \beta + sc \pmod{m} \\ x\alpha + xyc &\equiv \beta + sc \pmod{m} \end{aligned}$$

Raggruppando i termini che dipendono dalla sfida c , l'equazione diventa:

$$c(xy - s) \equiv \beta - x\alpha \pmod{m}$$

Poiché per ipotesi la quadrupla è falsa il termine $(xy - s)$ è diverso da zero. In un gruppo di ordine primo m , questo termine ammette un inverso moltiplicativo. Di conseguenza, esiste un unico valore di c che soddisfa l'uguaglianza:

$$c \equiv (\beta - x\alpha)(xy - s)^{-1} \pmod{m}$$

Tuttavia, l'attaccante deve inviare u e v (e quindi fissare α e β) al passo 1, prima che il Verifier scelga c al passo 2. Poiché c è scelto uniformemente a caso tra m valori possibili, la probabilità che l'attaccante abbia scelto i valori di impegno corretti per "indovinare" l'unica sfida possibile è esattamente:

$$Pr[\text{Successo dell'attaccante}] = \frac{1}{m}$$

Dato che m è un numero estremamente grande, questa probabilità è trascurabile, garantendo così la proprietà di Soundness.

- **Zero-Knowledge:** Cerchiamo innanzitutto di capire la view: (u, v) , c , z . Vediamo come costruire una tripletta a posteriori:

1. Scegliamo c random tra $1, \dots, m$
2. Scegliamo z random tra $0, \dots, m$
3. Calcoliamo $u = g^z / y^c$ e $v = X^z / W^c$. In questo modo, quando il Verificatore andrà a controllare se $X^z = v \cdot W^c$, l'uguaglianza sarà vera per costruzione, anche se il simulatore non sa assolutamente nulla di x o y .

Capitolo 6

Bitcoin Scripting

Le transazioni reali nella rete Bitcoin possiedono una struttura dati cruda (*raw data*) complessa e standardizzata, spesso rappresentata in formati simil-JSON per facilitarne la lettura da parte degli sviluppatori.



Come si evince dall'immagine, una transazione è divisa in tre sezioni logiche principali:

- **Metadati:** Informazioni di servizio generali come l'hash della transazione stessa (TXID), la versione del protocollo e il *locktime* che è il momento prima del quale la transazione non può essere inclusa in un blocco.
- **Inputs:** Contengono il riferimento ai fondi che si vogliono spendere.

- **prev_out**: È l'*hash pointer* che punta univocamente all'UTXO di una transazione precedente. Contiene il TXID precedente e l'indice (n) dello specifico output all'interno di quella transazione.
 - **scriptSig**: Conosciuto anche come **Unlocking Script**, è la componente crittografica che fornisce la prova di possesso dei fondi. Contiene la **firma digitale** e la **chiave pubblica di colui che possiede l'UTXO**.
- **Outputs**: Definiscono dove andranno i fondi e a quali condizioni.
 - **value**: La quantità di bitcoin trasferiti, espressa in *satoshi* (la più piccola unità di Bitcoin, pari a 10^{-8} BTC).
 - **scriptPubKey**: Conosciuto anche come **Locking Script**, contiene l'**indirizzo del destinatario** che non è altro che l'hash della sua chiave pubblica e, soprattutto, le condizioni logiche necessarie affinché questi fondi possano essere spesi in futuro.

L'utilizzo dei termini "script" non è casuale. Il controllo sulla spesa dei Bitcoin non è affidato a una semplice verifica di saldi, ma all'esecuzione di veri e propri algoritmi scritti in **Bitcoin Script**.

Firma di un Input e del TXID

Volendo riassumere in modo rigoroso il processo di firma e la definizione dell'identificativo della transazione, per ogni input che si desidera spendere avviene quanto segue:

1. **Preparazione (Svuotamento e Sostituzione)**: Si crea una copia temporanea della transazione in cui tutti i campi `scriptSig` vengono svuotati. Successivamente, esclusivamente nell'input che si sta validando, si inserisce temporaneamente lo `scriptPubKey` dell'UTXO di origine al posto di `scriptSig`.
2. **Hashing temporaneo e Firma**: Si calcola l'hash di questa transazione temporanea per ottenere il *Transaction Digest*. Questo hash rappresenta il "messaggio" che viene effettivamente firmato crittograficamente utilizzando la chiave privata dell'utente.
3. **Assemblaggio finale**: Si scarta l'hash temporaneo appena calcolato. La firma digitale ottenuta al passo precedente, accompagnata dalla chiave pubblica in chiaro, viene inserita definitivamente nel campo `scriptSig` della transazione *originale*.

4. **Calcolo del TXID e Trasmissione:** A questo punto la transazione è completa e pronta per essere trasmessa (in *broadcast*) alla rete. Il **TXID** (Transaction ID) viene calcolato dai nodi applicando un doppio SHA-256 all'intera transazione finale. È fondamentale ricordare che il TXID *non* viene inserito come campo all'interno dei dati della transazione, ma funge esclusivamente da "impronta digitale" usata dal network per identificarla univocamente nella blockchain.

6.1 Il linguaggio Bitcoin Script

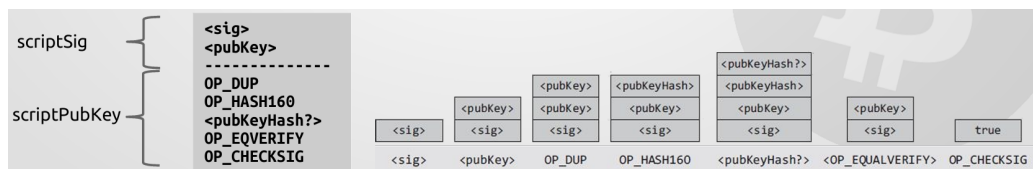
Bitcoin Script è un linguaggio di scripting semplice, basato su una struttura dati a **pila** (pila o *stack*). È progettato appositamente per essere sicuro e prevedibile:

- **Nessuna memoria (*stateless*):** L'esecuzione di uno script non ha memoria delle esecuzioni precedenti. Non esistono parametri di stato salvati.
- **Non Turing-completo:** Il linguaggio manca intenzionalmente di costrutti per cicli infiniti (come `for` o `while`), garantendo che ogni script termini la sua esecuzione in un tempo prevedibile, prevenendo attacchi di tipo *Denial of Service* (DoS) ai nodi della rete.

Con questo linguaggio di scripting è possibile effettuare controlli su **firme multiple con threshold** o **proof of burn**.

6.1.1 Il processo di validazione: Sblocco e Blocco

Il cuore della sicurezza di Bitcoin risiede nel meccanismo con cui un input valida la spesa di un output precedente. Per verificare se una transazione ha il diritto di spendere un determinato UTXO, il network non esegue un solo script, ma combina lo script di sblocco dell'input attuale con lo script di blocco dell'output precedente.



Come mostrato nel diagramma, il processo di valutazione standard (ad esempio per una transazione di tipo *Pay-to-Public-Key-Hash* o P2PKH) avviene seguendo questi passi logici:

1. **Esecuzione dell'Unlocking Script:** I nodi prendono lo **scriptSig** dall'input della transazione attuale. Questo script contiene la firma digitale e la chiave pubblica. Questi elementi vengono caricati (*push*) sulla pila uno dopo l'altro.
2. **Esecuzione del Locking Script:** Subito dopo, senza ripulire la pila, i nodi eseguono lo **scriptPubKey** prelevato dall'UTXO precedente che si vuole spendere. Questo script contiene una serie di operazioni logiche (opcodes) e l'hash dell'indirizzo del destinatario originale.
3. **Verifica Crittografica (Passi logici):** Durante l'esecuzione combinata, lo script esegue due controlli fondamentali per garantire che il mittente sia l'autentico proprietario dei fondi:
 - **Corrispondenza della Chiave Pubblica:** Lo **scriptPubKey** estrae la chiave pubblica fornita nello **scriptSig**, ne calcola l'hash e verifica che questo hash corrisponda esattamente all'hash dell'indirizzo salvato nello script di blocco originale. Questo assicura che la chiave pubblica fornita sia quella corretta.
 - **Verifica della Firma:** Infine, lo **scriptPubKey** utilizza la chiave pubblica appena verificata per validare la firma digitale crittografica (anch'essa fornita nello **scriptSig**). Questa firma deve essere stata creata matematicamente a partire dai dati della transazione attuale, dimostrando che il mittente possiede la chiave privata corrispondente e che la transazione non è stata alterata.

Se, al termine dell'esecuzione di entrambe le parti dello script combinato, la pila contiene un singolo elemento non nullo (interpretato come "Vero" o 1), la transazione è considerata crittograficamente valida e può essere propagata nella rete e inclusa in un blocco. In caso contrario, è scartata.

6.1.2 Multi Signatures con Threshold

Come abbiamo già visto, Bitcoin permette di spendere fondi provenienti da più UTXO a patto che si possa provare, tramite apposita firma, la proprietà di ognuno di essi. Abbiamo definito questo caso come un **pagamento congiunto** poiché, in termini pratici, gli UTXO possono appartenere a persone diverse e servono quindi le firme di tutti i coinvolti per autorizzare l'operazione.

Un caso particolare è il sistema di **Multi Signatures con Threshold**. Tramite questa operazione, facciamo sì che un singolo UTXO sia vincolato a un gruppo di chiavi pubbliche, ma possa essere speso fornendo solo un

numero minimo di firme (la soglia o *threshold*) rispetto al totale delle chiavi presenti. Ad esempio, una tipica multi-signature potrebbe elencare 5 chiavi pubbliche nell'UTXO, ma per spendere i fondi ne basterebbero solo 3, indipendentemente da quali siano i firmatari tra i 5 autorizzati.

Esiste inoltre la possibilità che un UTXO richieda la firma di tutti i suoi proprietari per poter essere speso. Possiamo definire questa situazione come un caso specifico di **Multi Signature con threshold N-su-N**, dove per muovere i fondi è necessaria la firma di tutti coloro che possiedono le chiavi associate all'UTXO, senza alcuna eccezione.

È chiaro che in questo scenario il rischio aumento poiché se anche una sola di queste chiavi viene persa, o se uno dei partecipanti si rifiuta di collaborare, l'UTXO diventa permanentemente bloccato e i fondi non potranno più essere recuperati in alcun modo.

6.1.3 Proof-of-Burn

La **Proof-of-Burn** è un'operazione mediante la quale è possibile letteralmente **buttare soldi**. Esistono principalmente due modi per farlo: il primo consiste nell'inviare i fondi a un cosiddetto **burn address**, ovvero un indirizzo di cui matematicamente nessuno possiede (o può generare) la chiave privata; il secondo è l'utilizzo dell'operazione *OP_RETURN*, che permette di contrassegnare un output come non spendibile a priori, consentendo però di aggiungere una piccola quantità di dati alla blockchain.

Questa è una pratica che nel contesto puro di Bitcoin ha poco senso, poiché ha come unico effetto quello di distruggere valuta. Tuttavia, viene utilizzata per scopi specifici come il **bootstrap** di una nuova moneta (dimostrando di aver "sacrificato" valore reale per ottenerla) o per il **timestam-ping**, ovvero usare la blockchain come un registro notarile per certificare l'esistenza di un dato in una certa data.

6.1.4 Timestamping

Il sistema di **Timestamping** permette di utilizzare la blockchain di Bitcoin come un registro notarile immutabile. Attraverso l'inserimento dell'hash di un documento all'interno di una transazione (spesso sfruttando l'operazione *OP_RETURN*), è possibile dimostrare con certezza assoluta che tale file esisteva già in una specifica data, certificata dal timestamp del blocco che contiene la transazione.

Inoltre, la flessibilità dello Script consente l'inserimento di quelli che vengono chiamati **Easter Eggs**: messaggi, citazioni o dati arbitrari nascosti all'interno della catena. Questa pratica, iniziata da Satoshi Nakamoto

con il messaggio nel blocco genesi, serve a lasciare una traccia indelebile di informazioni non finanziarie direttamente nel registro distribuito.

6.1.5 Real-life transaction con Escrow

Un problema classico derivante dal sistema Bitcoin è quello dei pagamenti online tra parti che non si fidano l'una dell'altra. Supponiamo che A voglia acquistare un bene da B : A non vuole rischiare di inviare i soldi e non ricevere nulla, mentre B non vuole spedire l'oggetto senza la garanzia di essere pagato. Questa situazione di stallo è nota in computer security come il problema dell'**Equa compravendita**. Per risolvere questa impasse, introduciamo una terza parte fidata, un **trusted third party** che chiameremo Giudice (J). La soluzione consiste nel creare una transazione con **multi signature a threshold 2-su-3** tra A , B e J . In questo modo i fondi vengono "bloccati" in un UTXO che richiede almeno due firme per essere speso:

- **Caso standard:** Se l'ordine arriva correttamente, A e B firmano insieme e i soldi vanno a B . Il giudice non interviene.
- **Caso di disputa:** Se nasce un problema, il giudice J analizza le prove. Se dà ragione ad A , firmerà insieme ad A per restituirgli i soldi (reso); se dà ragione a B , firmerà insieme a B per completare il pagamento.

L'aspetto fondamentale è che il giudice J , pur avendo il potere di decidere l'esito della disputa, non può mai scappare con i soldi da solo, poiché la soglia di firme richiesta è 2 e lui ne possiede soltanto una.

6.1.6 Micro-pagamenti e Lightning Network

Supponiamo che A e B vogliano scambiarsi pagamenti frequenti, ad esempio per un servizio di chiamata tariffato al minuto. Effettuare una transazione sulla blockchain per ogni singolo minuto sarebbe insostenibile: i tempi di conferma sono lunghi e le commissioni (fees) per i miner supererebbero probabilmente il valore del pagamento stesso. Per risolvere questo problema, sfruttiamo il **lock_time** combinato con una **multi-signature 2-su-2**. Il processo si divide in tre fasi:

1. **Apertura del canale:** A prepara una transazione di finanziamento (funding) che blocca una somma in un output richiedente le firme di entrambi (A e B). **Attenzione:** prima di pubblicare questa transazione, A si fa firmare da B una transazione di **rescue** (con un *lock_time* futuro) che gli restituirebbe l'intero importo se il canale dovesse bloccarsi.

2. **Scambio off-chain:** Durante la chiamata, *A* invia a *B* delle "micro-transazioni" cumulative. Queste transazioni sono firmate solo da *A*, quindi non sono ancora valide per la rete. Ad ogni minuto, *A* aggiorna la transazione assegnando a *B* una quota maggiore e a se stesso (come resto) una quota minore.
3. **Chiusura:** Alla fine della chiamata, *B* prende l'ultima transazione ricevuta (quella con l'importo totale corretto), appone la sua firma e la pubblica sulla blockchain.

B è tranquillo perché ha in mano una transazione che può riscuotere in qualsiasi momento. *A* è tranquillo perché, se *B* dovesse sparire per non farlo cooperare, basterebbe attendere la scadenza del *lock_time* per riprendersi i soldi tramite la transazione di rescue. Una volta che la transazione finale viene pubblicata da *B*, quella di rescue diventa automaticamente nulla, poiché tenterebbe di spendere un UTXO che è già stato consumato (evitando così il *double-spending*).

6.1.7 Freezing Funds

Supponiamo che *A* voglia mettere da parte una certa somma di Bitcoin per il futuro, ad esempio come fondo di risparmio per un figlio, oppure che voglia semplicemente "auto-imporsi" di non spendere i propri fondi per un determinato periodo di tempo. In una transazione standard, chiunque possieda la chiave privata può muovere i fondi in qualsiasi momento; tuttavia, Bitcoin permette di "congelare" letteralmente un UTXO grazie all'istruzione *OP_CHECKLOCKTIMEVERIFY*. Tramite questa operazione, facciamo sì che l'output della transazione non sia solo vincolato a una firma, ma anche a una condizione temporale specifica (espressa in data o in altezza del blocco). In pratica, lo script dell'UTXO dirà: "Questi soldi possono essere spesi solo se viene fornita la firma di *A* **E** se il tempo trascorso è superiore a *X*". Naturalmente, questa condizione è scritta sulla blockchain e verificata dai nodi, il che la rende inviolabile: anche se *A* possiede la chiave privata, se prova a spendere i fondi prima della scadenza, la rete rifiuterà la transazione considerandola invalida. Questa funzione è fondamentale non solo per il risparmio a lungo termine, ma anche per la sicurezza di protocolli complessi. Come abbiamo visto nei micro-pagamenti, il sistema di "congelamento" permette di creare delle vie d'uscita (le transazioni di *rescue*): se una controparte smette di collaborare, i fondi non restano bloccati per sempre nel limbo della multi-signature, ma ritornano automaticamente al proprietario originale una volta che il "tempo di blocco" è scaduto.

Gemini ha detto Certamente! Ho integrato i tuoi appunti di lezione mantenendo la struttura narrativa che abbiamo usato finora (problema → soluzione tecnica → vantaggio pratico).

Ecco la sezione sul P2SH per i tuoi appunti:

6.1.8 Pay-to-Script-Hash (P2SH)

Tutte le funzionalità che abbiamo analizzato finora, come le multi-signature o i sistemi di `lock_time`, hanno un "difetto" naturale: rendono lo script di pagamento estremamente lungo e complesso. Se un venditore B chiedesse a un acquirente A di pagare verso uno script così complicato, A si ritroverebbe a dover inserire una mole enorme di dati nell'output della transazione, pagando di conseguenza commissioni molto più alte ai miner.

Per risolvere questo problema, utilizziamo il sistema **Pay-to-Script-Hash (P2SH)**. In questo scenario, immaginiamo che gli script complessi siano gestiti interamente dal venditore B. Invece di inviare a A l'intero set di istruzioni, B crea prima lo script (chiamato *redeem script*), ne calcola l'**hash** e invia solo quest'ultimo ad A.

Questa procedura garantisce diversi vantaggi:

- **Efficienza:** Lo *scriptPubKey* rimane molto piccolo perché contiene solo l'hash dello script. Questo rende la transazione d'acquisto economica per A, indipendentemente dalla complessità delle regole imposte da B.
- **Sicurezza:** Il venditore deve generare l'hash e "pubblicarlo" (fornendolo ad A) prima del pagamento. In questo modo, B non può più modificare le condizioni dello script per provare a "barare", poiché se cambiasse anche una sola virgola dello script originale, l'hash non corrisponderebbe più e i fondi risulterebbero bloccati.

In pratica, i soldi rimangono bloccati nell'hash finché B non decide di spenderli. Solo in quel momento, all'interno dello *scriptSig*, B dovrà fornire sia lo script completo che tutte le firme necessarie per farlo girare. La rete Bitcoin verificherà prima che lo script corrisponda all'hash e poi eseguirà le istruzioni per autorizzare il pagamento.

Capitolo 7

La Blockchain

Una volta che le transazioni vengono create e firmate tramite la logica degli script, esse devono essere raggruppate e "sigillate" all'interno di un **blocco**. La blockchain non è altro che una catena sequenziale di questi contenitori, dove ogni blocco garantisce l'immutabilità dei dati che contiene.

7.1 Il Blocco

Ogni blocco è identificato da un **Header** (intestazione), che possiamo immaginare come la carta d'identità del blocco stesso. Esso contiene i metadati fondamentali per la sicurezza della rete:

- **Hash del blocco precedente (*prev_block*):** È il legame che crea la catena. Ogni blocco contiene l'impronta digitale di quello precedente; se qualcuno provasse a modificare una transazione in un blocco passato, l'hash cambierebbe, rendendo istantaneamente invalidi tutti i blocchi successivi.
- **Nonce e Bits:** Sono parametri legati al mining. Il *nonce* è il numero che i miner continuano a variare per trovare un hash che soddisfi la difficoltà della rete (*bits*), confermando così il blocco tramite la Proof-of-Work.
- **Merkle Root (*mrkl_root*):** Una stringa che riassume in modo univoco tutte le transazioni presenti nel blocco

7.1.1 Merkle Tree e Proof of Membership

Per gestire i dati delle transazioni in modo efficiente, Bitcoin non le elenca semplicemente una dopo l'altra, ma utilizza una struttura chiamata **Merkle**

Tree (albero di Merkle). Si tratta di un albero binario in cui si parte dagli hash delle singole transazioni alla base e, accoppiandoli, si sale di livello calcolando nuovi hash fino ad arrivare alla radice (**Merkle Root**). Questa struttura è fondamentale per la cosiddetta **Proof of Membership**. Supponiamo che un utente *A* utilizzi un wallet "leggero" sul suo smartphone. Grazie al Merkle Tree, *A* non deve scaricare tutte le migliaia di transazioni del blocco, ma gli basta ricevere solo una piccola serie di hash intermedi. Combinando questi hash con quello della propria transazione, *A* può ricalcolare la Merkle Root: se il risultato coincide con quello presente nell'header del blocco, allora ha la certezza matematica che la sua transazione è stata confermata. Infatti, non tutti i partecipanti alla rete scaricano l'intera blockchain. Esistono due tipologie di nodi:

- **Full Node:** Sono i nodi che contengono l'intera cronologia delle transazioni e verificano ogni singola regola del protocollo. Sono "pesanti" in termini di memoria ma garantiscono la massima sicurezza. I miner possono solamente essere dei nodi di questo tipo.
- **Lightweight Node (o nodi SPV):** Sono nodi che non contengono tutte le transazioni, bensì conservano solo i **Block Header**. Quando questi nodi hanno necessità di verificare l'esistenza di una certa transazione, interrogano i Full Node chiedendo loro di inviargli i dati necessari (il "cammino" nell'albero di Merkle) per effettuare la Proof of Membership.

7.1.2 Coinbase transaction

Fino ad ora abbiamo visto come *A* possa inviare Bitcoin a *B* partendo da fondi già esistenti (UTXO). Ma come entrano in circolazione i nuovi Bitcoin se non esiste una banca centrale? La soluzione è la **Coinbase Transaction**, una transazione "speciale" che deve essere presente obbligatoriamente come prima operazione di ogni singolo blocco. Questa transazione ha tre caratteristiche fondamentali:

- **Emissione Monetaria:** È l'unico modo previsto dal protocollo per **creare nuovi Bitcoin**. Non avendo un input che proviene da una transazione precedente, essa "genera" valuta dal nulla seguendo una regola matematica precisa.
- **Remunerazione dei Miner:** Rappresenta il guadagno principale per chi mette in sicurezza la rete. Il miner che riesce a chiudere il blocco assegna a se stesso una somma composta da:

1. Il **Block Subsidy** (il premio per il blocco).
2. Le **Fees** (le commissioni) di tutte le transazioni incluse nel blocco.

Il premio iniziale era di 50 BTC, ma ogni 210.000 blocchi (circa 4 anni) questa cifra viene dimezzata tramite un evento chiamato **Halving**. Come si vede nell'immagine, nel 2020 il premio è sceso a 6.25 BTC e dall'aprile 2024 è passato a **3.125 BTC**.

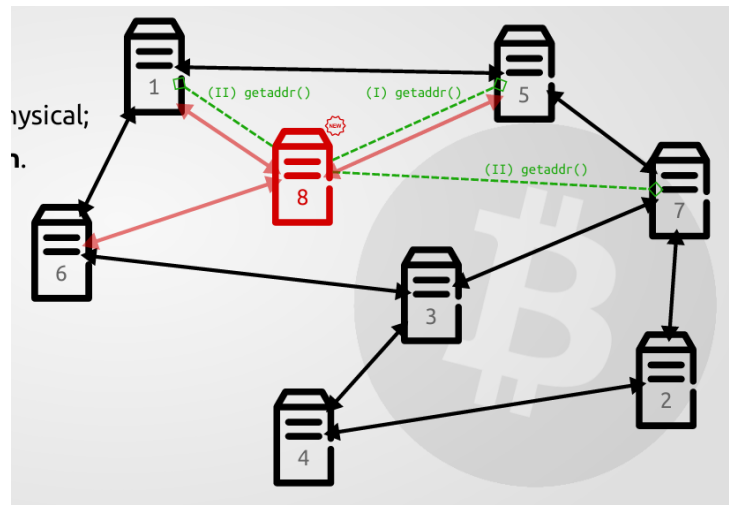
- **Il campo Coinbase:** A differenza delle transazioni normali, l'input della Coinbase Transaction non punta a un UTXO precedente (infatti il campo *hash* è composto da soli zeri). Al suo posto troviamo un campo libero chiamato *coinbase*, dove il miner può inserire dati arbitrari. L'esempio più famoso è quello di Satoshi Nakamoto nel Blocco 0, dove inserì il titolo del quotidiano *The Times* per denunciare l'instabilità del sistema bancario tradizionale.

7.2 Il Network Bitcoin

Dopo aver analizzato la struttura dei blocchi, è fondamentale capire come le informazioni (transazioni e blocchi) viaggino tra i partecipanti. Bitcoin non si appoggia a un server centrale, ma utilizza una rete **P2P ad-hoc** dove tutti i nodi sono considerati uguali.

La rete Bitcoin presenta alcune proprietà fondamentali che ne garantiscono la resilienza:

- **Connessioni TCP:** I nodi comunicano solitamente sulla porta 8333.
- **Topologia Logica vs Fisica:** La rete è organizzata in modo "random". Un nodo in Italia potrebbe essere logicamente connesso a un nodo in Giappone; la struttura logica non rispecchia quella geografica.
- **Discovery e Oblivion:** Quando un nuovo nodo entra nella rete (come il nodo 8 nell'immagine), deve "scoprire" i suoi vicini. Lo fa inviando messaggi di *getaddr()* per ottenere liste di indirizzi IP di altri nodi attivi. Allo stesso modo, se un nodo smette di rispondere, viene dimenticato dalla rete (*oblivion*).



7.2.1 Inserimento di una Transazione

Quando un utente *A* crea una transazione per pagare *B*, questa non finisce immediatamente nella blockchain. Deve prima essere propagata a tutti i nodi della rete tramite un **Gossip Protocol** (o *Flooding*). Il funzionamento è simile al diffondersi di un pettegolezzo: *A* invia la transazione ai propri vicini; questi la verificano e, se valida, la inviano ai propri vicini, e così via fino a coprire l'intera rete in pochi secondi.

7.2.2 La Transaction Pool e i Controlli Standard

Ogni singolo nodo mantiene in memoria una "vasca" temporanea chiamata **Transaction Pool** (o *Mempool*), dove conserva le transazioni ricevute che sono in attesa di essere inserite in un blocco dai miner. Prima di accettare una transazione nella propria pool e inoltrarla agli altri, ogni nodo effettua dei **controlli standard**:

1. **Validità:** La transazione deve avere firme corrette e formattazione valida.
2. **Script Whitelist:** Lo script deve rientrare tra quelli considerati "standard".
3. **Unicità:** La transazione non deve essere già presente nella pool.
4. **Assenza di conflitti:** La transazione non deve tentare di spendere UTXO già impegnati da altre transazioni presenti nella pool (evitando il double spending immediato).

7.2.3 Race Conditions e Coerenza delle Pool

Poiché la rete è distribuita e la propagazione richiede tempo, le Mempool dei vari nodi non sono sempre identiche (*out-of-sync*). Se due transazioni in conflitto (ad esempio *A* prova a mandare gli stessi soldi sia a *C* che a *B* contemporaneamente) si diffondono nella rete, si crea una **Race Condition**. In questo caso vale la regola del **"the first win!"**: ogni nodo terrà in memoria solo la prima transazione che riceve e scarterà la seconda. Sarà poi il miner, scegliendo quale includere nel prossimo blocco, a decretare quale delle due transazioni diventerà definitiva. Esiste però un'eccezione chiamata **Replace-By-Fee (RBF)**, che permette a un utente di "sovrascrivere" una propria transazione in attesa con una nuova che offra una mancia (*fee*) più alta ai miner.